WAF Arena: defensor y atacante de inyección SQL con aprendizaje por refuerzo

Diego Valenzuela, Daniel Dubón, Joaquín Puente, Nelson García

Reinforcement Learning, Universidad del Valle de Guatemala · Versión v1.0

Índice

1. Introducción 2
- Problema y motivación . . . 2
- Pregunta de investigación . . . 3
- Alcance . . . 3
- Contribuciones de la Fase 1 . . . 3

2. Trabajo relacionado 4
- Evasión de WAF por mutación . . . 4
- Aprendizaje por refuerzo aplicado a inyección SQL . . . 4
- Algoritmos de RL usados como herramientas . . . 4
- Self-play y entrenamiento contra adversarios propios . . . 4
- Lo que este capítulo no afirma . . . 5

3. Laboratorio y formulación 6
- Arquitectura . . . 6
- Oráculo . . . 6
- Corpus y tráfico benigno . . . 6
- Formulación como MDP/POMDP . . . 6
- Entorno de dos velocidades . . . 7
- Protocolo experimental y estadística . . . 7

4. Atacante clásico y RQ1 9
- Pregunta . . . 9
- Métodos evaluados . . . 9
- Resultados por experimento . . . 9
- Tabla de vulnerabilidad de ModSecurity (RQ1) . . . 10
- Limitaciones y hallazgos relevantes . . . 11
- `ATK-REF-0` . . . 11
- Figuras del capítulo . . . 12

5. Deep RL: defensor DQN, atacante de policy gradient y RQ2 16
- Pregunta . . . 16
- Protocolo propio de la fase . . . 16
- Defensor lineal (EXP-E1) y triada mortal (EXP-E2) . . . 16

DQN: prueba de cordura y ablación (EXP-F1 a EXP-F4) . . . 17
Frontera de Pareto contra ModSecurity (EXP-F5) . . . 17
Generalización del defensor estático (EXP-F6, EXP-F7) . . . 18
Qué aprende el defensor: ablaciones de observación (ABL-1, ABL-2) . . . 19
Atacante de policy gradient (EXP-G1 a EXP-G3) y entropía (EXP-G4) . . . 19
Latencia (F3-T10) . . . 20
Referencias publicadas . . . 20
Respuesta a RQ2 . . . 20
Limitaciones de esta fase . . . 21
Figuras del capítulo . . . 22

6. Planning y self-play 24
Preguntas . . . 24
Diseño experimental . . . 24
Resultados de self-play (RQ3) . . . 24
Resultados de Dyna (RQ4) . . . 25
Limitaciones de la fase . . . 26
Figuras del capítulo . . . 26

7. Generalización 29
Validación del modo rápido frente al laboratorio real . . . 29

8. Ética y alcance 30
Aislamiento . . . 30
Origen del corpus . . . 30
Propósito y política de publicación . . . 30
Limitación del oráculo en producción . . . 30

9. Conclusiones 31
Respuestas a las preguntas de investigación . . . 31
Síntesis . . . 31
Limitaciones consolidadas . . . 32
Trabajo futuro . . . 33

Referencias 34

1. Introducción

Problema y motivación

Los *Web Application Firewalls* (WAF) basados en reglas, como ModSecurity con el OWASP Core Rule Set (CRS), son la primera línea de defensa contra la inyección SQL. Su eficacia depende de reglas escritas a mano que un atacante puede eludir reescribiendo el mismo payload de formas semánticamente equivalentes. La pregunta que motiva este proyecto es si un atacante y un defensor que aprenden por refuerzo pueden mejorar, respectivamente, la evasión y la detección, y qué revela ese enfrentamiento sobre la robustez de un WAF de reglas.

Pregunta de investigación

La pregunta general y las cinco preguntas de investigación (RQ1–RQ5) están definidas en docs/00-vision/vision.md. En resumen:

- RQ1: ¿Qué tan vulnerable es ModSecurity (PL1–PL4) a un atacante de mutación clásico (bandits y métodos tabulares)?
- RQ2: ¿Mejora un defensor con aprendizaje profundo (DQN) la frontera entre bloqueo de ataques y falsos positivos frente a ModSecurity fijo?
- RQ3: ¿El self-play con un *pool* de oponentes produce políticas más generales que el entrenamiento contra un oponente fijo?
- RQ4: ¿Qué aporta la planificación (Dyna) frente a métodos libres de modelo con el mismo presupuesto de consultas?
- RQ5: ¿Cuánto generalizan los agentes a familias de ataque y a tráfico benigno difícil no vistos, y cuál es la brecha entre el modo rápido y el modo real?

Alcance

El alcance está cerrado a inyección SQL contra PostgreSQL (ADR-0001). XSS queda como extensión posterior a la entrega final.

Contribuciones de la Fase 1

Esta fase no responde ninguna RQ; construye la base sobre la que las demás fases experimentan:

1. Un laboratorio Docker aislado (`internal: true`, sin salida a internet) con una aplicación vulnerable propia, PostgreSQL instrumentada, cuatro instancias de ModSecurity (PL1–PL4), un proxy defensor y un honeypot.
2. Un oráculo verificado que decide con certeza si un payload es funcional y si tuvo efecto, mediante la huella estructural de la consulta (`pglast`) y canarios.
3. Un corpus de 528 payloads funcionales verificados y 5 200 sesiones de tráfico benigno, con *splits* congelados.
4. Dieciséis operadores de mutación, los dos entornos Gymnasium (`Attacker-v0`, `Defender-v0`) y los *backends* rápido y real.

Los detalles de diseño y verificación están en el capítulo 3.

2. Trabajo relacionado

Este capítulo ubica el proyecto frente a la literatura que lo motivó. Solo cita fuentes cotejadas en referencias verificadas. Los resultados propios no se discuten aquí; están en los capítulos 4 a 7.

Evasión de WAF por mutación

WAF-A-MoLE (`demetrio2020wafamole`) trata la evasión de un WAF como un problema de búsqueda: parte de un payload malicioso y le aplica operaciones que preservan su semántica hasta que el WAF deja de bloquearlo. La idea de mutar sin romper la función del ataque es la base del espacio de acciones del atacante de este proyecto (16 operadores, capítulo 3), y el oráculo del laboratorio cumple el papel de verificar que la función se conserve.

Este proyecto se diferencia en tres puntos. Primero, el atacante no es una búsqueda por ataque individual sino una política aprendida con métodos de RL de distintas familias (bandits, programación dinámica, Monte Carlo, TD, policy gradient y planificación; capítulos 4 a 6). Segundo, el WAF atacado, ModSecurity con el OWASP Core Rule Set (`owaspcrs`) en cuatro niveles de paranoia, se compara además con un defensor que también aprende. Tercero, la evasión se mide con presupuesto de consultas reales, no solo con tasa de éxito.

Aprendizaje por refuerzo aplicado a inyección SQL

Erdődi, Sommervoll y Zennaro (`erdodi2021sqlq`) modelan la explotación de una inyección SQL como un reto de captura de bandera formulado como proceso de decisión de Markov y entrenan agentes de Q-learning en un escenario sintético simplificado. Es el antecedente más cercano en cuanto a formular SQLi como un problema de RL. Su escenario es una simulación del reto de explotación; este proyecto, en cambio, usa una aplicación vulnerable real en un laboratorio aislado, un WAF real como adversario y un oráculo basado en la consulta ejecutada y en canarios (ADR-0004).

Algoritmos de RL usados como herramientas

Los métodos del curso siguen a Sutton y Barto (`sutton2018`): bandits, programación dinámica, Monte Carlo, diferencias temporales y aproximación de funciones. Para el defensor profundo se usó DQN (`mnih2015dqn`) y las variantes evaluadas en la ablación del capítulo 5: Double DQN (`vanhasselt2016double`), arquitectura Dueling (`wang2016dueling`) y *Prioritized Experience Replay* (`schaul2016per`). El atacante de gradiente de política parte de REINFORCE (`williams1992reinforce`) y de PPO (`schulman2017ppo`) con estimación de ventaja generalizada (`schulman2016gae`); el contexto de métodos actor-critic asíncronos es `mnih2016a3c`. Las implementaciones siguen el estilo de archivo único de CleanRL (`huang2022cleanrl`, ADR-0006) y los entornos exponen la API de Gymnasium (`gymnasium`).

La planificación del atacante (capítulo 6) usa la arquitectura Dyna (`sutton1990dyna`), que combina aprendizaje con experiencia real y actualizaciones sobre un modelo aprendido. En este proyecto el modelo es un sustituto supervisado de la respuesta del defensor, y solo las consultas reales consumen presupuesto.

Self-play y entrenamiento contra adversarios propios

AlphaZero (`silver2017alphazero`) es la referencia habitual de aprendizaje por self-play, donde el agente mejora jugando contra versiones de sí mismo. Bansal et al. (`bansal2018emergent`) estudian la competencia entre agentes como fuente de comportamientos complejos. Ambos trabajos justifican la pregunta

de RQ3, pero ninguno de los dos es un entorno de seguridad ni asimétrico como este: aquí atacante y defensor tienen acciones, observaciones y ritmos de entrenamiento distintos, y la comparación relevante es entre alternancia ingenua y entrenamiento contra un pool de generaciones anteriores. Este capítulo no usa esas fuentes para afirmar que el pool deba funcionar; el resultado propio, que no muestra una superioridad robusta, está en el capítulo 6.

Lo que este capítulo no afirma

No se hace una revisión sistemática de la literatura sobre WAF con aprendizaje automático ni sobre RL ofensivo. Las fuentes de este capítulo son las verificadas hasta la fecha de entrega; la cobertura es deliberadamente acotada y no sustituye una revisión exhaustiva.

3. Laboratorio y formulación

Arquitectura

El laboratorio corre en Docker Compose sobre una red declarada `internal: true`, sin salida a internet y sin puertos publicados al host. Una prueba automática verifica que ningún contenedor resuelve ni alcanza un host externo. Los componentes son: la aplicación vulnerable `vulnapp` (FastAPI), PostgreSQL con canarios, cuatro instancias de ModSecurity + OWASP CRS (una por paranoia level), un proxy defensor y un honeypot. El diagrama está en docs/01-arquitectura/arquitectura.md.

Se usa una aplicación propia en lugar de una de terceros (ADR-0007) porque permite saber con exactitud qué consulta ejecuta cada endpoint, lo que hace el oráculo trivial de especificar. `vulnapp` expone ocho endpoints vulnerables por concatenación de cadenas, catalogados en `lab/vulnapp/ENDPOINTS.md`, más flujos benignos.

Oráculo

El oráculo responde dos preguntas sobre un payload: si es funcional (el SQL resultante es válido y cambió la estructura de la consulta) y si tuvo efecto verificable (ADR-0004). Compara la huella de `libpg_query` (vía `pglast`) de la consulta ejecutada con la de la plantilla del endpoint; una huella distinta indica un cambio estructural. Los canarios aportan evidencia más fuerte: un token aleatorio de 32 caracteres generado en cada arranque, una tabla de escrituras y una fila oculta. Cada consulta ejecutada se registra en `oracle_log` con un rol de base de datos separado al de la aplicación.

Las cinco garantías del contrato se prueban automáticamente: todos los payloads de referencia dan `effect = True`; los valores benignos dan `effect = False`; un payload sin la comilla de cierre no es funcional; el mismo payload evaluado cien veces da el mismo resultado; y ninguna consulta legítima devuelve el token del canario.

Corpus y tráfico benigno

El corpus (`data/payloads/corpus.jsonl`) contiene 528 payloads funcionales verificados por el oráculo, por encima del mínimo de 200 y de 30 por familia: tautology 120, boolean_blind 120, error 196, union 52, stacked 40. El tráfico benigno (`data/benign/sessions.jsonl`) contiene 5 200 sesiones (1 200 difíciles, 240 por cada etiqueta de dificultad), todas con `effect = False` según el oráculo. Ambos conjuntos se generan de forma reproducible (mismo hash al regenerar).

Los *splits* (`data/splits/v1.json`) se congelan en esta fase. Las familias `error` y `stacked` se reservan completas para `test-family`; el resto se divide 60/20/20 por grupo base, de modo que las variantes de un mismo payload no cruzan particiones. Tamaños: payloads 172/56/64/236 (train/val/test/test-family); benignos 2 832/944/944/480 (train/val/test/test-hard). La regla `make verify-splits` recalcula los hashes y falla si los archivos cambian.

Formulación como MDP/POMDP

Atacante (**`Attacker-v0`**). Un episodio es un intento de evadir al defensor partiendo de un payload base. La observación es un vector de 288 dimensiones (hashing de n-gramas de caracteres, ventana de las últimas respuestas, contadores de presupuesto y familia). Las acciones son `Discrete(17)`: aplicar uno de los 16 operadores de mutación o `SEND`. La verificación de funcionalidad tras cada mutación usa el oráculo sin WAF y no consume consultas; esta es una simplificación declarada: equivale a que el atacante tenga una copia local del objetivo. El atacante no observa el estado interno del defensor, solo sus respuestas, por

lo que el problema es un POMDP; la observación incluye una ventana de las últimas cuatro respuestas como aproximación.

Defensor (**`Defender-v0`**). Un episodio es una sesión de tráfico; cada paso es un request. La observación es un vector de 37 dimensiones (features del request, los cuatro scores de anomalía de ModSecurity, features de sesión y una ventana). Las acciones son `Discrete(5)`: permitir, bloquear, captcha, limitar o desviar al honeypot. Incluir los cuatro scores de ModSecurity hace del defensor un sistema híbrido (ADR-0003).

Para métodos tabulares, el atacante expone un estado abstracto biyectivo en `[0, 120)` combinando familia, última respuesta y un *bucket* de mutaciones.

Entorno de dos velocidades

El entrenamiento necesita cientos de miles de pasos, inviables por HTTP (ADR-0005). El modo rápido (`FastBackend`) ejecuta las consultas del oráculo en proceso y obtiene los scores de ModSecurity por HTTP con una caché persistente por `(versión de configuración, PL, hash del request)`; el modo real (`RealBackend`) hace todo por HTTP. Con caché caliente el modo rápido sostiene 3.27 millones de pasos/s para la verificación de funcionalidad y 234 000 pasos/s incluyendo los scores de ModSecurity, muy por encima de la meta de 300 pasos/s. La concordancia entre ambos modos es $\geq$ 99 % sobre los requests muestreados.

El anomaly score por request se obtiene del audit log JSON de ModSecurity, correlacionado por `X-Request-ID` en un volumen compartido (ADR-0008).

Protocolo experimental y estadística

La Fase 1 deja la prueba de humo como línea base de cordura: un atacante aleatorio contra ModSecurity PL1–PL4 obtiene una tasa de evasión de 0.0 en los cuatro niveles, en modo rápido y real (`results/summary/smoke.csv`), lo que confirma que la evasión no es trivial y motiva el aprendizaje. El resto de esta sección (runner, semillas y estadística) lo completa la Fase 2.

Runner. Todo experimento corre con `make run CONFIG=configs/exp/<exp_id>/<run_name>.yaml` (`wafarena/run.py`, contrato de experimentos), dentro del contenedor `runner` del laboratorio. El YAML se valida contra un esquema (claves desconocidas son error), se rechaza cualquier `env.split` distinto de `train` para entrenar, y el `exp_id` debe existir en el catálogo. El runner se niega a correr con cambios sin commit en `wafarena/` o `configs/` salvo `--allow-dirty`, que queda marcado en `meta.yaml` y no vale para el informe. Cada corrida produce `results/<exp_id>/<run_name>/s<seed>/` con `config.resolved.yaml`, `meta.yaml` (sha de git, rama, duración, versiones de contratos), `train.csv` (una fila por episodio), `eval.csv` (una fila por punto de evaluación en `val`), `final_eval.json` y el mejor `checkpoint/` en `val`. Dos corridas con la misma semilla producen un `eval.csv` idéntico. Los métodos de programación dinámica y los bandits usan runners especializados (`wafarena/dp_run.py`, `wafarena/bandit_run.py`) con el mismo formato de salida: el primero porque no tiene bucle de entrenamiento por pasos (estima el modelo y resuelve una sola vez), el segundo porque opera por tiradas, no por episodios con mutaciones.

Semillas. Mínimo 5 semillas por configuración principal; 10 para bandits y métodos tabulares, por ser baratos de correr. Todas las configuraciones de EXP-A1 a EXP-D2 y de la tabla RQ1 usan 10 semillas (`seeds: [0..9]` en el YAML), con un backend y una caché compartidos entre semillas dentro de una misma corrida para evitar reconstruir el estado del laboratorio en cada una.

Estadística. Las cifras del informe son la media y el intervalo de confianza del 95 % por *bootstrap* sobre semillas (10 000 remuestreos, `wafarena/eval/stats.py:bootstrap_ci`), nunca un valor puntual. Para

afirmar que un método supera a otro se reporta el IC de la diferencia; si contiene el 0, el informe dice que no hay diferencia detectable — es el caso de todos los métodos de este capítulo, empatados en ER = 0,000 (capítulo 4). Las métricas con denominador vacío (por ejemplo QFE sin ninguna evasión) se reportan como `nan` y quedan fuera del *bootstrap*. Los resúmenes versionados (`make summarize EXP=<exp_id>` → `results/summary/<exp_id>.csv`) y las figuras (`report/figuras/make_<exp_id>.py`) se regeneran siempre desde los CSV crudos de cada corrida, nunca a mano.

4. Atacante clásico y RQ1

Pregunta

RQ1: ¿qué tan vulnerable es ModSecurity (PL1–PL4) a un atacante de mutación que aprende con los métodos clásicos (bandits, programación dinámica, Monte Carlo, SARSA, Q-learning, SARSA(λ))? El protocolo de evaluación (runner, semillas, estadística) está en el capítulo 3, «Protocolo experimental y estadística».

Métodos evaluados

Todos operan sobre el estado abstracto `[0, 120)` de `Attacker-v0` (familia × última respuesta × *bucket* de mutaciones) salvo los bandits, que ven solo el payload base como un brazo entre 16.

Experimento	Método	Variantes
EXP-A1	*k*-armed bandits	greedy, ε-greedy (0,01 / 0,1), optimista, UCB (c = 0,5 / 2), aleatorio
EXP-A2	*k*-armed bandits	ε-greedy 0,1 con paso constante (α = 0,05 / 0,1 / 0,3) contra promedio muestral
EXP-B1	Programación dinámica	*value iteration* y *policy iteration* sobre un MDP estimado con 200 000 transiciones
EXP-C1	Monte Carlo	control ε-soft, *first-visit* y *every-visit* (ε = 0,05 / 0,1)
EXP-D1	TD de un paso	SARSA y Q-learning, ε = 0,1 fijo
EXP-D2	TD(λ)	SARSA(λ) con trazas de elegibilidad reemplazantes, λ ∈ {0, 0,5, 0,9}

Resultados por experimento

EXP-A1/A2 — Bandits. Los 7 métodos de EXP-A1 y las 4 configuraciones de EXP-A2 corrieron 10 semillas cada uno contra el laboratorio real (110 corridas, 0 fallos). ER = 0,000 [0,000; 0,000] en todos; el regret acumulado en las 20 000 tiradas es 0 para los 7 métodos de EXP-A1, y EXP-A2 no tiene nada que recuperar tras el cambio de defensor porque ya era 0 antes y después (`results/summary/EXP-A1.csv`, `EXP-A2.csv`). La causa está aguas arriba del método de selección: la media estimada *fuera de línea* de los 16 brazos contra `modsec-pl1…pl4` es 0 (`results/summary/bandit_arm_means.csv`, 2 000 tiradas por brazo), y una búsqueda voraz de 25 pasos con 64 candidatos por paso no logra bajar la puntuación de ningún payload de `train` del umbral de bloqueo (`results/summary/evasion_reachability_pl1.csv`). Ningún algoritmo de selección de brazos puede explotar una señal que no existe.

EXP-B1 — Programación dinámica. Con el modelo estimado (200 000 transiciones, política aleatoria, `modsec-pl1`), *value iteration* converge en 1 147 iteraciones y *policy iteration* en 3; ambas coinciden en 114 de los 120 estados abstractos, y los 6 restantes son empates exactos de valor Q entre acciones igual de inútiles (verificado a mano), no un desacuerdo real (`results/summary/EXP-B1.csv`). La política óptima bajo este modelo aproximado tiene ER = 0,000 [0,000; 0,000] en `attacker-eval-v1`: ni la solución exacta del MDP estimado evade. Esto es una cota bajo un modelo aproximado — el estado abstracto no distingue el conteo de bloqueos acumulados, así que el modelo mezcla transiciones terminales y no terminales en los estados con bloqueo parcial — pero coincide con el resultado directo de EXP-A1.

EXP-C1 — Monte Carlo. *First-visit* y *every-visit*, ε ∈ {0,05; 0,1}, 200 000 pasos, 10 semillas, laboratorio real (40 corridas). ER = 0,000 [0,000; 0,000] en los 4 métodos, curva plana en `val` durante todo el entrenamiento (`results/summary/EXP-C1.csv`, `report/figuras/EXP-C1_er_vs_steps.png`).

EXP-D1 — SARSA y Q-learning. $\varepsilon = 0{,}1$ fijo, `pl1` y `pl2`, 200 000 pasos, 10 semillas (70 corridas con EXP-D2). ER = 0,000 [0,000; 0,000] en los 4 métodos (`results/summary/EXP-D1.csv`). Se midió además la tasa de rotura del payload (BRK) durante el entrenamiento, buscando el efecto *cliff* clásico (Q-learning más conservador que SARSA cerca del «precipicio»): aquí ocurre lo contrario — SARSA rompe más (BRK final 0,0533) que Q-learning (0,0396), y la brecha es consistente durante las 20 ventanas de entrenamiento (`report/figuras/EXP-D1_sarsa_vs_qlearning.png`). La hipótesis del catálogo no se confirma: en *cliff walking* el camino arriesgado es también el de mayor recompensa, y aquí no — toda mutación cuesta −0,1 sin importar qué tan cerca esté de romper el payload, así que no hay asimetría de riesgo/recompensa que premie al método más conservador.

EXP-D2 — SARSA(λ). $\lambda \in \{0; 0{,}5; 0{,}9\}$ contra `pl1`. ER = 0,000 [0,000; 0,000] en los 3 valores, plana desde el primer punto de `val` (`results/summary/EXP-D2.csv`, `report/figuras/EXP-D2_er_vs_steps.png`). La métrica propuesta en el catálogo (pasos hasta igualar la ER final de SARSA de EXP-D1) es degenerada aquí: ambos lados de la comparación son 0.

Tabla de vulnerabilidad de ModSecurity (RQ1)

Para responder RQ1 directamente se evaluó el mejor checkpoint de cada familia contra las cuatro instancias de ModSecurity, comparado con las dos referencias guionizadas de la Fase 1 (`results/summary/RQ1_table.csv`, 10 semillas × 8 métodos × 4 PL = 32 celdas):

Método	PL1	PL2	PL3	PL4
bandit-greedy	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]
dp	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]
monte-carlo	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]
sarsa	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]
qlearning	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]
sarsa-lambda	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]
corpus-replay	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]
random-mutation	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]	0,000 [0,000; 0,000]

(Media [IC 95 % por bootstrap]; valores exactos en `results/summary/RQ1_table.csv`.) QFE, QFE-c y DIV quedan sin definir (`nan`) en las 32 celdas por falta de evasiones de las que calcularlas. El método representante de cada familia no se eligió por comparación entre ellos — las 8 filas están empatadas en 0 — sino por ser la configuración más estándar de cada uno: `bandit-greedy` (única configuración sin ventaja frente a las demás de EXP-A1/A2), `dp` (única configuración de EXP-B1), `monte-carlo` = *first-visit* $\varepsilon = 0{,}1$, `sarsa` y `qlearning` = `pl1`, `sarsa-lambda` = $\lambda = 0{,}9$.

Respuesta a RQ1

Bajo el laboratorio actual, con los 16 operadores de mutación del registro congelado (operadores-mutacion.md), ModSecurity no es vulnerable a ningún atacante clásico de las semanas 2–6 del curso — bandits, programación dinámica, Monte Carlo, SARSA, Q-learning ni SARSA(λ) — en ningún nivel de paranoia (PL1–PL4). El techo está en el espacio de ataque (los operadores disponibles), no en el método de aprendizaje: la evidencia de EXP-A1 es directa y no depende de ningún algoritmo de RL — la media de recompensa de los 16 brazos contra las 4 instancias de ModSecurity es exactamente 0, y una búsqueda voraz (no aprendida) de 25 pasos tampoco encuentra una sola mutación que baje la puntuación de un payload del umbral de bloqueo. Ningún método posterior (DP, MC, TD) puede aprender una política mejor que lo que el espacio de mutación permite, y en efecto ninguno lo hace: las 32 celdas de la tabla

son idénticas a 0. Esto es una respuesta negativa pero completa a RQ1 dentro del alcance declarado de esta fase, no un resultado inconcluso.

Limitaciones y hallazgos relevantes

- El resultado depende de los 16 operadores actuales, no de un límite teórico de ModSecurity. Es consistente con el riesgo documentado en el plan de esta fase («es un resultado válido si se mide bien») y queda abierto como trabajo futuro ampliar el espacio de mutación (por ejemplo, con un operador de cambio de ubicación del payload que sí modifique lo que reciben los *backends*, a diferencia de `location_switch` hoy).
- Despliegue voraz y sesgo de optimismo. Al preparar `ATK-REF-0` (F2-T10) se encontró que, al desplegarse de forma puramente voraz (`explore=False`), las políticas de DP, Monte Carlo, SARSA y SARSA(λ) casi nunca llegan a `SEND`: con la tabla Q inicializada en cero y toda recompensa real negativa, una acción nunca visitada siempre parece mejor que cualquiera ya explorada, así que la política se queda mutando indefinidamente (verificado contra el laboratorio real: 148–150 de 150 pasos mutando, 0–2 consultas enviadas). Esto no invalida ER = 0,000 en ninguna evaluación de este capítulo — un episodio que nunca envía tampoco evade — pero sí significa que, desplegados tal cual fuera de la evaluación controlada, estos atacantes casi no atacan. Por eso `ATK-REF-0` es un bandit (`greedy-pl1`, EXP-A1, semilla 0): ataca por protocolo fijo (alterna mutar y enviar), no por valor de Q aprendido, y es el único que se comporta como oponente real dentro de `Defender-v0`.
- Dos bugs de infraestructura encontrados bajo carga sostenida en el laboratorio real (crecimiento sin cota del operador `hex_string` reaplicado sobre sí mismo; lectura del log de auditoría de ModSecurity desde el byte 0 en cada proceso nuevo) se corrigieron antes de registrar los runs afectados; no cambian ninguna cifra de este capítulo. Detalle en la sección *Hallazgos* de fase-2-daniel-atacante-clasico.md.

`ATK-REF-0`

Se publica `checkpoints/refs/ATK-REF-0/` (contrato agentes-y-checkpoints.md): el checkpoint de `greedy-pl1` (EXP-A1, semilla 0), elegido por ser el único método que ataca de forma confiable al desplegarse (ver arriba), no por ER en `val` — todos empatan en 0. Se agregó a la suite `defender-eval-v1` (contrato de métricas, versión 1.2) a través de `AttackerDriver` (`wafarena/attackers/driver.py`), que reconstruye la observación de 288 dimensiones y el estado abstracto dentro de `Defender-v0` sin oráculo ni *backend*. Verificado contra el laboratorio real y como prueba unitaria: `load_checkpoint("checkpoints/refs/ATK-REF-0")` funciona y `AttackerDriver` lo conduce sin errores en 100 sesiones de `Defender-v0`.

Figuras del capítulo

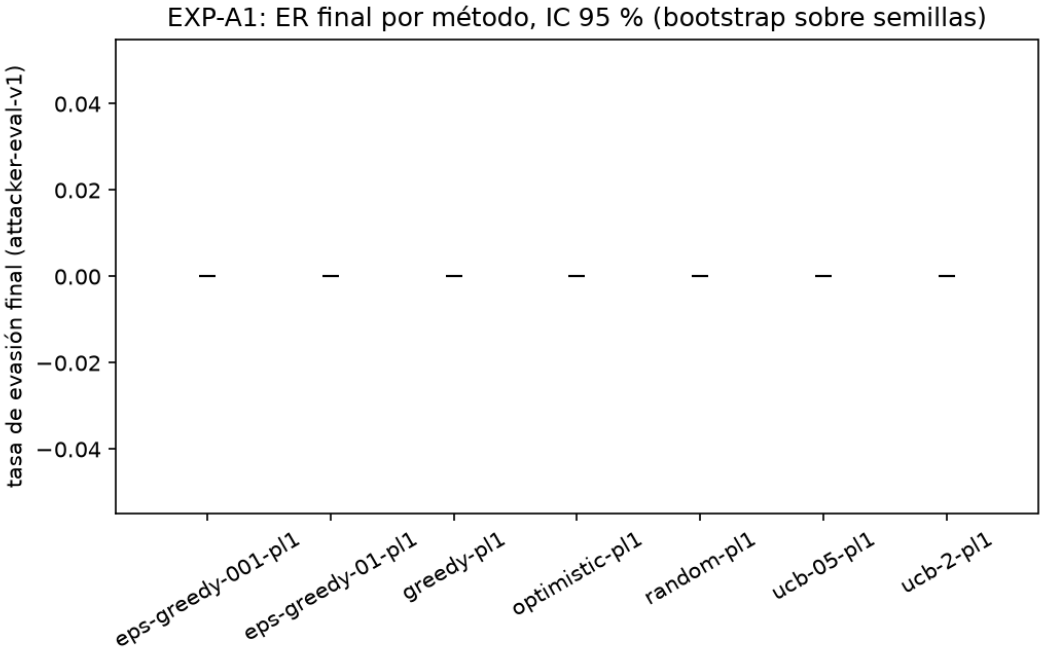


Figura 1: EXP-A1: recompensa media de los bandits.

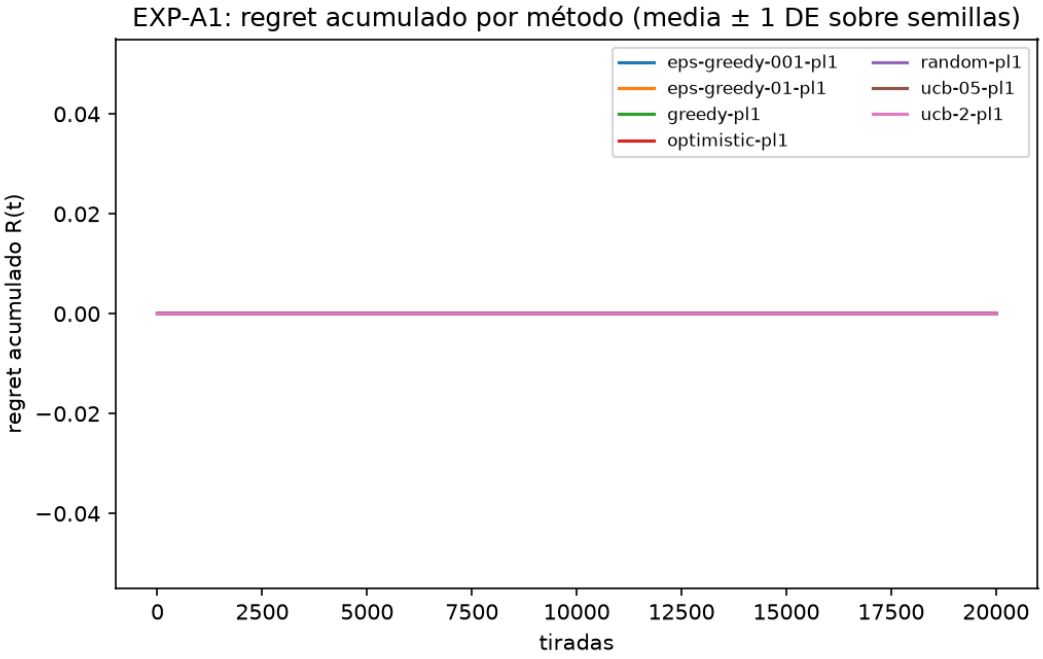


Figura 2: EXP-A1: regret acumulado de los bandits.

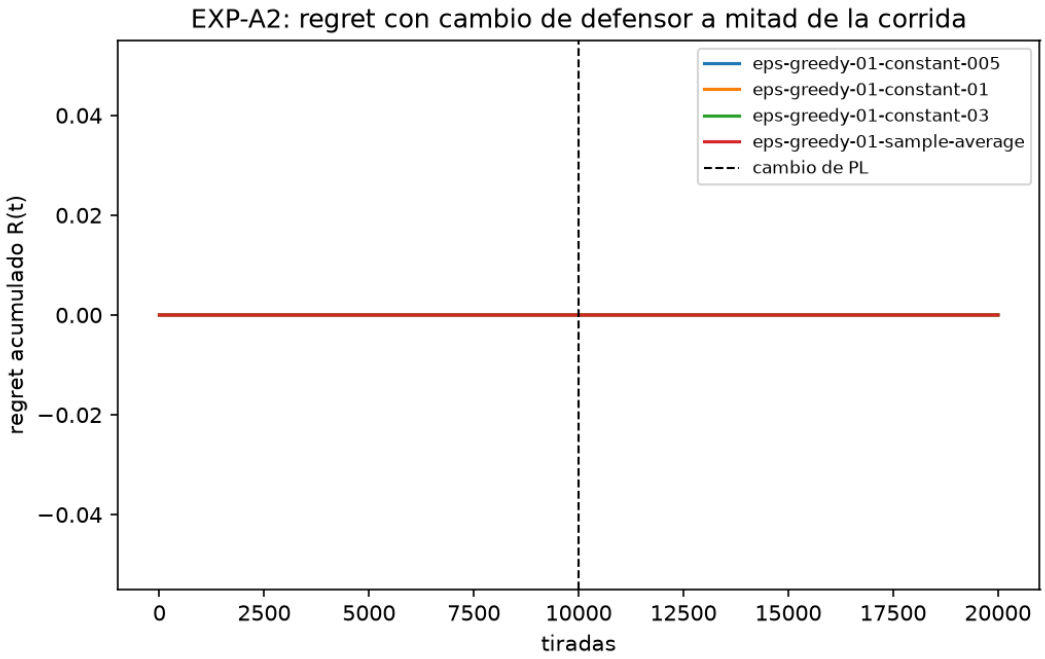


Figura 3: EXP-A2: recuperación ante cambio de nivel de paranoia.

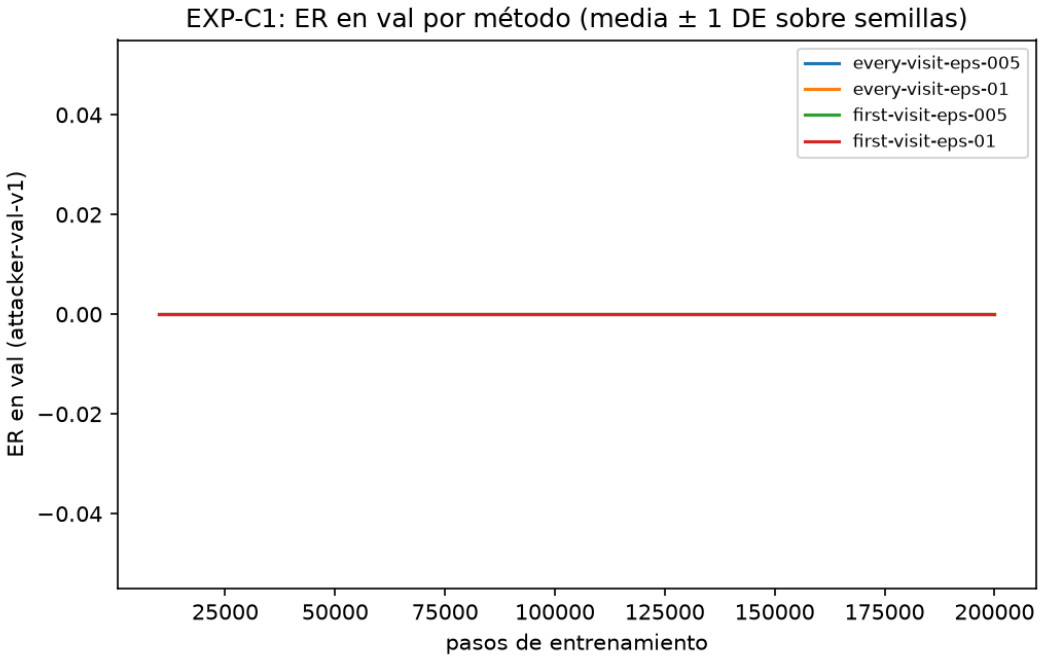


Figura 4: EXP-C1: tasa de evasión contra pasos de entrenamiento.

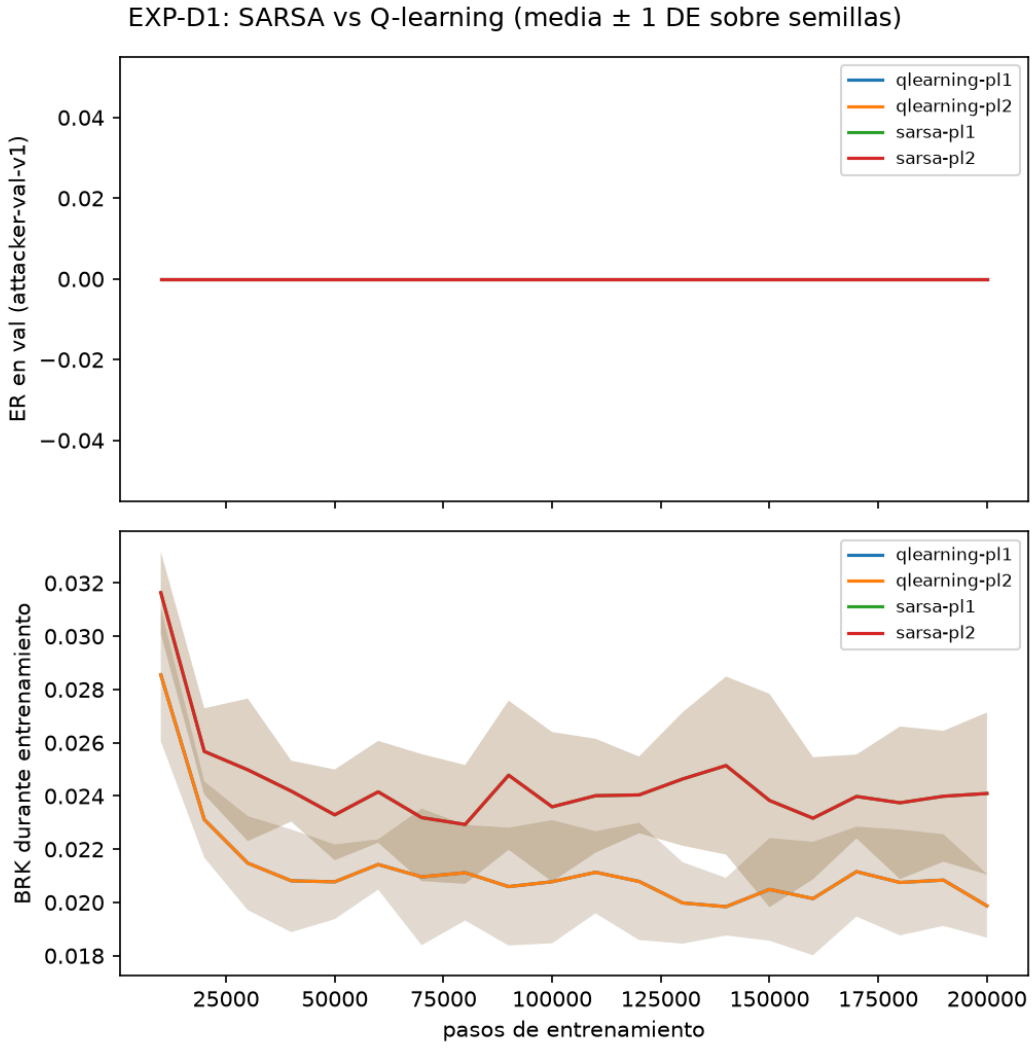


Figura 5: EXP-D1: SARSA frente a Q-learning.

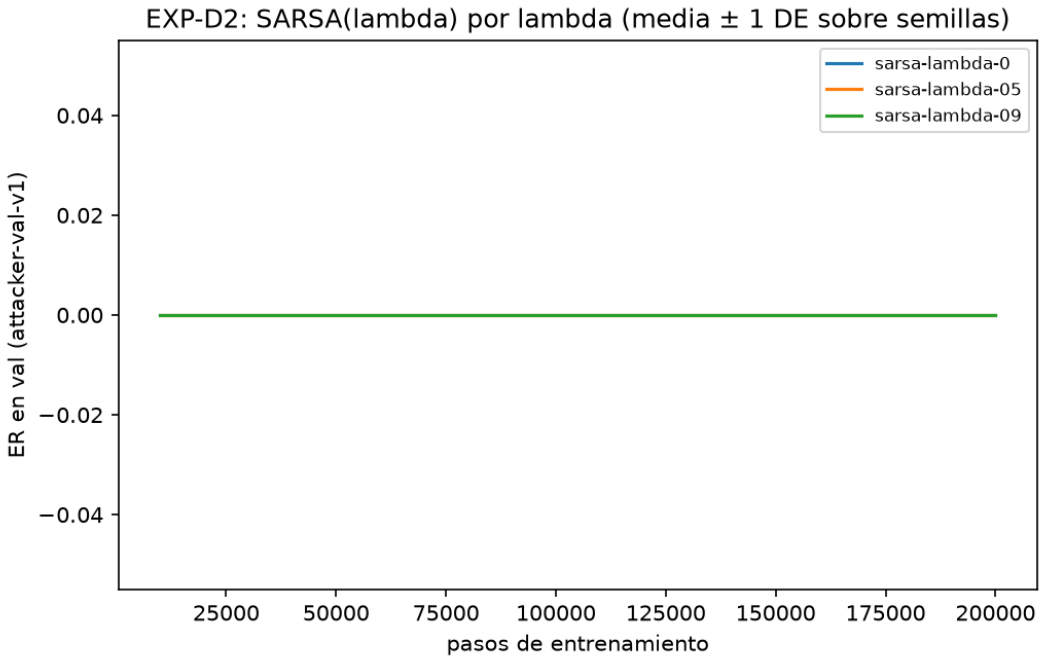


Figura 6: EXP-D2: tasa de evasión contra pasos de entrenamiento.

5. Deep RL: defensor DQN, atacante de policy gradient y RQ2

Pregunta

RQ2: ¿puede un defensor entrenado con RL competir con ModSecurity en la frontera entre detección y falsos positivos, y qué tan fuerte es un atacante de policy gradient contra él? Este capítulo responde además la parte de RQ5 que toca al defensor *estático* (entrenado contra oponentes fijos): cuánto generaliza a familias de ataque y a tráfico benigno que no vio. Esa es la línea base contra la que la Fase 4 compara al defensor de *self-play*.

El protocolo general (runner, semillas, IC por *bootstrap*) es el del capítulo 3. Lo que agrega esta fase se resume en la sección siguiente.

Protocolo propio de la fase

- Defensor en el runner. `Defender-v0` corre con `make run` desde esta fase. Salvo que el experimento diga otra cosa, el defensor entrena contra un *pool* uniforme de los tres atacantes fijos (`corpus-replay`, `random-mutation`, `ATK-REF-0`) con `attack_ratio = 0,1`; cada sesión de ataque sortea su atacante con el RNG del entorno.
- Selección de modelo en **`val`**. Se agregó la suite `defender-val-v1` (2 000 sesiones del split `val`). El checkpoint de cada run es el punto de evaluación de mayor TPR en `val` sujeto a FPR en `val` $\leq$ FPR de `modsec-pl2` en `val`; ese tope (`fpr_cap`) lo mide el runner al empezar. **`fpr_cap`** = 0,0: `modsec-pl2` no tiene ni un falso positivo en las sesiones benignas de `val` (`results/summary/modsec_defender_suites.csv`). La regla exige entonces FPR = 0 exacto en `val`.
- Agregación de una suite. Una suite del defensor se juega una vez por atacante; las métricas se calculan sobre todos los requests juntos (contrato de métricas 1.3) y además por atacante (`tpr@<atacante>`).
- Experimentos solo de evaluación. EXP-F6 y EXP-F7 no entrenan: vuelven a evaluar checkpoints de EXP-F5 en otras suites (`wafarena.evaluate`, contrato de experimentos 1.3). Cada resultado conserva el `run_id` del run que entrenó el checkpoint.
- ModSecurity como referencia. ModSecurity no aprende y las suites son fijas, así que PL1–PL4 se evalúan una vez y no tienen IC.

Defensor lineal (EXP-E1) y triada mortal (EXP-E2)

EXP-E1. Q-learning y SARSA semi-gradiente con Q(s, a) = w_a · [features, 1], 100 000 pasos, 5 semillas, contra el *pool* por defecto. En `defender-eval-v1`: Q-learning TPR = 0,955 [0,895; 0,991], FPR = 0,016 [0,011; 0,019], F1 = 0,935 [0,915; 0,947]; SARSA TPR = 0,950 [0,879; 0,991], FPR = 0,022 [0,016; 0,028], F1 = 0,919 [0,892; 0,940] (`results/summary/EXP-E1.csv`). Ninguno de los 50 puntos de evaluación (2 métodos × 5 semillas × 10 evaluaciones) llega a FPR = 0 en `val`, así que ninguno cumple la regla de selección. Un modelo lineal no basta para separar sin falsos positivos.

EXP-E2. Aproximación lineal + *bootstrapping* + datos *off-policy* de una política uniforme fija (Q-learning, $\alpha = 0{,}05$, $\gamma = 0{,}99$), contra SARSA *on-policy* con los mismos α y γ, 3 semillas. La norma de los pesos de la configuración *off-policy* crece sin estabilizarse, de $\approx$ 19–22 a los 5 000 pasos a 232–309 a los 100 000, y el error TD medio sube a 8,5–11,8. La *on-policy* se estabiliza en $\|w\| \approx 50$ con $|\delta| \approx 1{,}1$ (`results/summary/EXP-E2_weight_norm.csv`, `report/figuras/EXP-E2_weight_norm.png`). Es el comportamiento que predice la triada mortal (Sutton y Barto, sección 11.3). En 100 000 pasos se ve como crecimiento sostenido, no como un desborde a infinito.

DQN: prueba de cordura y ablación (EXP-F1 a EXP-F4)

Prueba de cordura. Antes de usar DQN en el laboratorio se verificó en CartPole-v1 (60 000 pasos, 3 semillas por variante). Cada 5 000 pasos se guarda el mejor *snapshot* en episodios de selección, y ese *snapshot* se confirma en 20 episodios distintos; aprueba con retorno medio ≥ 195. Aprobaron 12 de 12 (`results/summary/dqn_cartpole_sanity.csv`). El primer criterio probado, la política *final* tras 60 000 pasos, aprobó solo 6 de 12 con el mismo código: DQN en CartPole aprende y luego oscila. Por ejemplo, Dueling semilla 0 llega a 314 a los 15 000 pasos y termina en 19,5. Ese criterio medía la estabilidad del final del entrenamiento, no si la implementación aprende, y se reemplazó sin tocar ningún hiperparámetro.

Ablación. Cuatro variantes con los mismos hiperparámetros ($\alpha = 4$, 150 000 pasos), cada una con una mejora más que la anterior, 5 semillas (`results/summary/EXP-F_ablation.csv`, `report/figuras/EXP-F_ablation.png`):

Variante	TPR	FPR	F1	Q-ALLOW	ataques que pasaron en el *batch*
base (EXP-F1)	0,9970 [0,9943; 0,9998]	0,0000 [0,0000; 0,0000]	0,9985	0,162 [−0,005; 0,340]	0,0058 [0,0056; 0,0061]
+ Double (EXP-F2)	0,9955 [0,9925; 0,9986]	0,0000 [0,0000; 0,0000]	0,9978	0,122 [−0,046; 0,277]	0,0059 [0,0057; 0,0063]
+ Dueling (EXP-F3)	0,9984 [0,9967; 0,9994]	0,0002 [0,0000; 0,0005]	0,9987	0,199 [0,130; 0,304]	0,0055 [0,0053; 0,0058]
+ PER (EXP-F4)	0,9828 [0,9740; 0,9917]	0,0000 [0,0000; 0,0000]	0,9913	0,208 [0,003; 0,365]	0,0281 [0,0257; 0,0303]

Q-ALLOW está en unidades de recompensa escalada (`reward_scale = 0,1`). Ninguna de las tres hipótesis del catálogo se confirma con el IC de la diferencia (`results/summary/EXP-F_ablation_diffs.csv`):

- Double no reduce de forma detectable la Q de `ALLOW` en ataques: −0,041 [−0,288; 0,195].
- Dueling no cambia el TPR de forma detectable: +0,003 [−0,0004; 0,006].
- PER sí cumple su mecanismo: repite unas 5 veces más ataques que pasaron. Pero baja el TPR, −0,016 [−0,025; −0,007], y el F1, −0,007 [−0,012; −0,003].

Con TPR $\approx$ 0,99 sin PER casi no quedan ataques que se escapen, y sobremuestrear esos pocos no ayuda.

Ya el DQN base supera de lejos al lineal: la red no lineal separa sin falsos positivos donde el lineal no puede. La mejor variante en `val` bajo la regla de selección es Double + Dueling (TPR medio en `val` 0,99929, 5/5 semillas con FPR = 0), y es la que usa EXP-F5. Se eligió en `val`; `test` no participó.

Frontera de Pareto contra ModSecurity (EXP-F5)

Double + Dueling con $\alpha \in \{1, 2, 4, 8, 16\}$, $\beta = \alpha/4$ y $\gamma = \alpha/8$, 5 semillas por α, en `defender-eval-v1` (`results/summary/EXP-F5.csv`, `report/figuras/EXP-F5_pareto.png`):

Defensor	TPR	FPR
DQN $\alpha = 1$	0,9809 [0,9741; 0,9896]	0,0005 [0,0000; 0,0015]
DQN $\alpha = 2$	0,9958 [0,9918; 0,9986]	0,0002 [0,0000; 0,0004]
DQN $\alpha = 4$	0,9984 [0,9967; 0,9994]	0,0002 [0,0000; 0,0005]
DQN $\alpha = 8$	0,9968 [0,9922; 0,9995]	0,0000 [0,0000; 0,0000]
DQN $\alpha = 16$	0,9987 [0,9979; 0,9994]	0,0000 [0,0000; 0,0000]

Defensor	TPR	FPR
modsec-pl1	0,7150	0,0000
modsec-pl2	0,9725	0,0000
modsec-pl3	1,0000	0,3327
modsec-pl4	1,0000	0,3844

La dominancia se decide de forma conservadora, con todo el IC: un α domina a un PL solo si todo su IC de TPR queda en o sobre el TPR del PL y todo su IC de FPR en o bajo el FPR del PL, con al menos una desigualdad estricta (`results/summary/EXP-F5_dominance.csv`).

- PL1 y PL2 quedan dominados por $\alpha = 8$ y $\alpha = 16$: mismo FPR $= 0$ y más detección en todo el IC.
- PL3 y PL4 no quedan dominados: detectan el 100 % de los ataques y ningún α llega a TPR $= 1$ en todo su IC. Pagan por eso un FPR de 0,33–0,38, dos órdenes de magnitud sobre el del DQN.

El barrido no produce el compromiso que se esperaba: el FPR ya es ≈ 0 desde $\alpha = 1$ y el TPR no es monótono en α. La frontera del DQN es casi un punto en FPR ≈ 0, no una curva.

Selección de **`DEF-REF-1`**. 24 de los 25 checkpoints cumplen la regla y 7 empatan exactamente con TPR $=$ 1,0 y FPR $=$ 0,0 en `val`. El empate se rompe solo con `val` (ADR-0010): menor FPR, mayor F1, mayor TPR medio en `val` de su α y menor semilla. **`DEF-REF-1`** = EXP-F5 **`alpha-4`**, semilla 0 (`results/summary/DEF-REF-1_selection.csv`, `checkpoints/refs/DEF-REF-1/`).

Generalización del defensor estático (EXP-F6, EXP-F7)

Las 5 semillas de la configuración de `DEF-REF-1` (EXP-F5 `alpha-4`) se evaluaron en las dos suites con datos no vistos (`results/summary/EXP-F6F7_generalization.csv`):

Suite	Defensor	TPR	FPR	FPR-h
`defender-eval-family-v1` (familias no vistas)	DQN	1,0000 [1,0000; 1,0000]	0,0002 [0,0000; 0,0005]	0,0000
	`modsec-pl1` / `pl2`	0,8547 / 0,9926	0,0000 / 0,0000	0,0000 / 0,0000
	`modsec-pl3` / `pl4`	1,0000 / 1,0000	0,3327 / 0,3844	0,3327 / 0,3844
`defender-eval-hard-v1` (benignos difíciles)	DQN	0,9997 [0,9992; 1,0000]	0,3385 [0,2775; 0,3996]	0,0583 [0,0000; 0,1333]
	`modsec-pl1` / `pl2`	0,9113 / 0,9846	0,0000 / 0,0000	0,0000 / 0,0000
	`modsec-pl3` / `pl4`	1,0000 / 1,0000	0,5105 / 1,0000	0,5105 / 1,0000

- Familias no vistas: el DQN generaliza. Atacante por atacante su TPR sigue en 1,0, igual que en `defender-eval-v1`.
- Benignos difíciles: el DQN no generaliza. Su FPR sube de ≈ 0 a 0,34; el cambio respecto a `defender-eval-v1` es +0,338 [+0,277; +0,399]. Queda peor que PL1 y PL2, que no tienen falsos positivos, y mejor que PL3 (0,51) y PL4 (1,00). La mayoría de esos falsos positivos son acciones blandas (captcha, *rate limit*): el FPR-h es 0,058.

Esta es la debilidad que la Fase 4 debería atacar. Una nota de lectura: las dos suites usan solo los atacantes con guion (sin `ATK-REF-0`), así que un TPR agregado no es comparable directamente con el de `defender-eval-v1`; la comparación justa es atacante por atacante.

Qué aprende el defensor: ablaciones de observación (ABL-1, ABL-2)

La configuración de DEF-REF-1 (EXP-F5 alpha-4) se reentrenó con una máscara de observación configurable por YAML, sin tocar el contrato del entorno: la máscara vive en el agente y viaja con el checkpoint. Son 5 semillas por configuración, en defender-eval-v1 (results/summary/ABL_comparison.csv):

Configuración	TPR	Diferencia de TPR con la completa	F1
Completa (37 features)	0,9984 [0,9967; 0,9994]	—	0,9987 [0,9976; 0,9997]
ABL-1: sin scores de ModSecurity (12–15)	0,9982 [0,9974; 0,9991]	−0,0002 [−0,0017; +0,0017]	0,9991 [0,9987; 0,9996]
ABL-2: sin sesión ni ventana (16–36)	1,0000 [1,0000; 1,0000]	+0,0016 [+0,0006; +0,0033]	1,0000 [1,0000; 1,0000]

El DQN no depende de ModSecurity. Sin sus scores detecta igual: las 12 features del request bastan para separar los ataques de este corpus del tráfico benigno. Por eso dominar a modsec-pl2 en EXP-F5 no consiste en copiar un nivel de paranoia. Quitar la sesión y la ventana incluso mejora un poco, de forma detectable: en estas suites cada request se decide por sí mismo y la historia no aporta. Esto matiza la motivación del defensor híbrido (ADR-0003) solo en lo que estas suites pueden medir.

Atacante de policy gradient (EXP-G1 a EXP-G3) y entropía (EXP-G4)

EXP-G1 a EXP-G3. REINFORCE con línea base, A2C y PPO con GAE sobre la observación de 288 dimensiones, 200 000 pasos y 5 semillas, entrenados y evaluados contra modsec-pl1 y contra DEF-REF-1, en attacker-eval-v1 (results/summary/EXP-G1.csv a EXP-G3.csv):

Atacante	ER contra modsec-pl1	ER contra DEF-REF-1
REINFORCE	0,0000 [0,0000; 0,0000]	0,0040 [0,0020; 0,0050]
A2C	0,0000 [0,0000; 0,0000]	0,0000 [0,0000; 0,0000]
PPO	0,0000 [0,0000; 0,0000]	0,0010 [0,0000; 0,0030]
ATK-REF-0 (referencia)	0,000	0,000

ATK-REF-0 viene de results/summary/ATK-REF-0_eval.csv; es una sola evaluación porque el checkpoint y la suite son fijos. Contra ModSecurity PL1 ningún método evade, lo mismo que con los atacantes clásicos del capítulo 4: con los 16 operadores el techo sigue estando en el espacio de ataque. Contra DEF-REF-1 hay evasiones raras. Las de REINFORCE tienen un IC que excluye el 0, así que supera a ATK-REF-0, pero se trata de menos de un episodio de 200 por semilla. QFE vale 10 en todas las semillas con evasión, y DIV queda indefinida porque ninguna semilla tiene dos payloads exitosos distintos.

El punto ciego de la consulta 10. Todas las evasiones contra DEF-REF-1 caen exactamente en la décima consulta de la sesión. Un diagnóstico exploratorio (scripts/diag_defref1_session.py, results/summary/DEF-REF-1_session_diag.csv) muestra que la política del DQN cambia con la longitud de la sesión justo ahí: frente al mismo payload sin mutar bloquea las consultas 1 a 9 y desde la 10 pasa a HONEYPOT. Una explicación posible, no verificada, es que en Defender-v0 los atacantes del *pool* abandonan tras 10 bloqueos, así que el DQN casi no vio sesiones de ataque más largas y en el punto de transición algunos payloads mutados reciben ALLOW. Es el tipo de debilidad que un oponente que se adapta (Fase 4) debería encontrar.

EXP-G4. PPO contra DEF–REF–1 con coeficiente de entropía 0, 0,01 y 0,05, 5 semillas (results/summary/EXP–G4_entropy.csv, report/figuras/EXP–G4_div_er.png). El bono sí vuelve más estocástico al atacante: la entropía de la política al final del entrenamiento pasa de 0,33 [0,18; 0,49] a 0,68 [0,48; 0,89] y a 2,19 [2,13; 2,26], de un máximo de ln 17 ≈ 2,83. El ER no cambia de forma detectable: 0,0010 [0,0000; 0,0030], 0,0010 [0,0000; 0,0030] y 0,0000. Con estos datos no se puede afirmar que un atacante más estocástico sea más difícil de bloquear. Con una evasión cada 1 000 episodios la pregunta no se puede responder en este espacio de ataque, y DIV no existe en ninguna configuración.

Latencia (F3-T10)

DEF–REF–1 servido por el proxy en modo real, con 2 000 requests del tráfico de defender–eval–v1, cada uno enviado a modsec–pl2 solo y a través del proxy, en orden alternado y con la máquina sin otra carga (results/summary/latency.csv):

Medida	p50	p95
LAT (agregado sobre ModSecurity solo)	65,6 ms	74,3 ms
modsec–pl2 solo	15,9 ms	17,9 ms
Proxy con DEF–REF–1	81,7 ms	90,3 ms
Decisión del DQN sola	0,034 ms	0,042 ms

Casi toda la latencia agregada es del proxy y no de la red neuronal. El proxy consulta ModSecurity PL1–PL4 en secuencia para armar las features 12–15 antes de decidir, del orden de cuatro veces una consulta sola. Como ABL-1 muestra que el DQN no necesita esos scores, un defensor sin ellos, o un proxy que consulte los cuatro niveles en paralelo, bajaría la latencia casi a la de ModSecurity solo.

Referencias publicadas

- **DEF–REF–1**: EXP-F5 alpha–4, semilla 0 (DQN Double + Dueling, $\alpha = 4$), elegido con la regla del plan más el desempate de ADR-0010, todo en val.
- **ATK–REF–1**: EXP-G3 ppo–defref1, semilla 1, el mejor PPO por ER en val contra DEF–REF–1; tiene ER = 0,005 en val.

Las dos cargan con load_checkpoint y funcionan como oponentes en 100 episodios (results/summary/refs_check.c
Son el punto de partida del *self-play* de la Fase 4.

Respuesta a RQ2

Sí: un defensor entrenado con RL compite con ModSecurity en la frontera entre detección y falsos positivos, y la mejora en una parte de ella.

- En defender–eval–v1, el DQN (Double + Dueling) domina a ModSecurity PL1 y PL2: con el mismo FPR = 0 detecta más ataques, con TPR ≈ 0,997–0,999 contra 0,715 y 0,973, comparando con el IC completo.
- PL3 y PL4 no quedan dominados, porque detectan el 100 %. Sin embargo, pagan un FPR de 0,33–0,38, mientras que el DQN tiene un FPR del orden de 10^{-4}.
- El DQN no imita a ModSecurity: sin sus scores detecta igual (ABL-1).
- La ventaja tiene dos límites: el defensor estático no generaliza a tráfico benigno difícil no visto (FPR 0,34 en defender–eval–hard–v1), y cuesta ~66 ms de latencia agregada por cómo está construido el proxy.

- Del lado del atacante, los métodos de *policy gradient* no son más fuertes que los tabulares contra ModSecurity (ER = 0 los tres). Contra el DQN sí encuentran una grieta, aunque estrecha: evasiones raras y siempre en la consulta 10.

Limitaciones de esta fase

- Las suites son fáciles para el DQN. TPR y F1 $\approx$ 0,99 dejan poco espacio para distinguir variantes. Por eso la ablación de DQN no separa Double de la base ni Dueling de Double, y el barrido de α no dibuja una curva.
- **`fpr_cap`** = 0. La regla de selección exige FPR = 0 exacto en `val`, porque `modsec-pl2` no tiene falsos positivos ahí. Eso deja muchos empates, que se resolvieron con ADR-0010.
- El espacio de ataque limita a los atacantes. Con los 16 operadores, ningún atacante evade ModSecurity, así que las preguntas sobre el atacante (EXP-G4, la DIV) apenas tienen señal. Es el mismo techo del capítulo 4.
- Entorno sin vectorizar. El riesgo «PPO lento» del plan no se materializó: un run de PPO de 200 000 pasos tarda 303 s solo en la máquina, y uno de DQN de 150 000 pasos 202 s (corridas de verificación del handoff, idénticas en `final_eval.json` a las de la campaña con la misma semilla). Se entrenó con un solo entorno, sin entornos vectorizados.
- Incidentes de infraestructura durante los runs, corregidos o mitigados y documentados en los *Hallazgos* de la Fase 3:
  - una colisión de nombres de temporales en la caché, porque todo contenedor tiene PID 1;
  - el disco lleno por los *audit logs* de ModSecurity, que nunca se rotan.

Figuras del capítulo

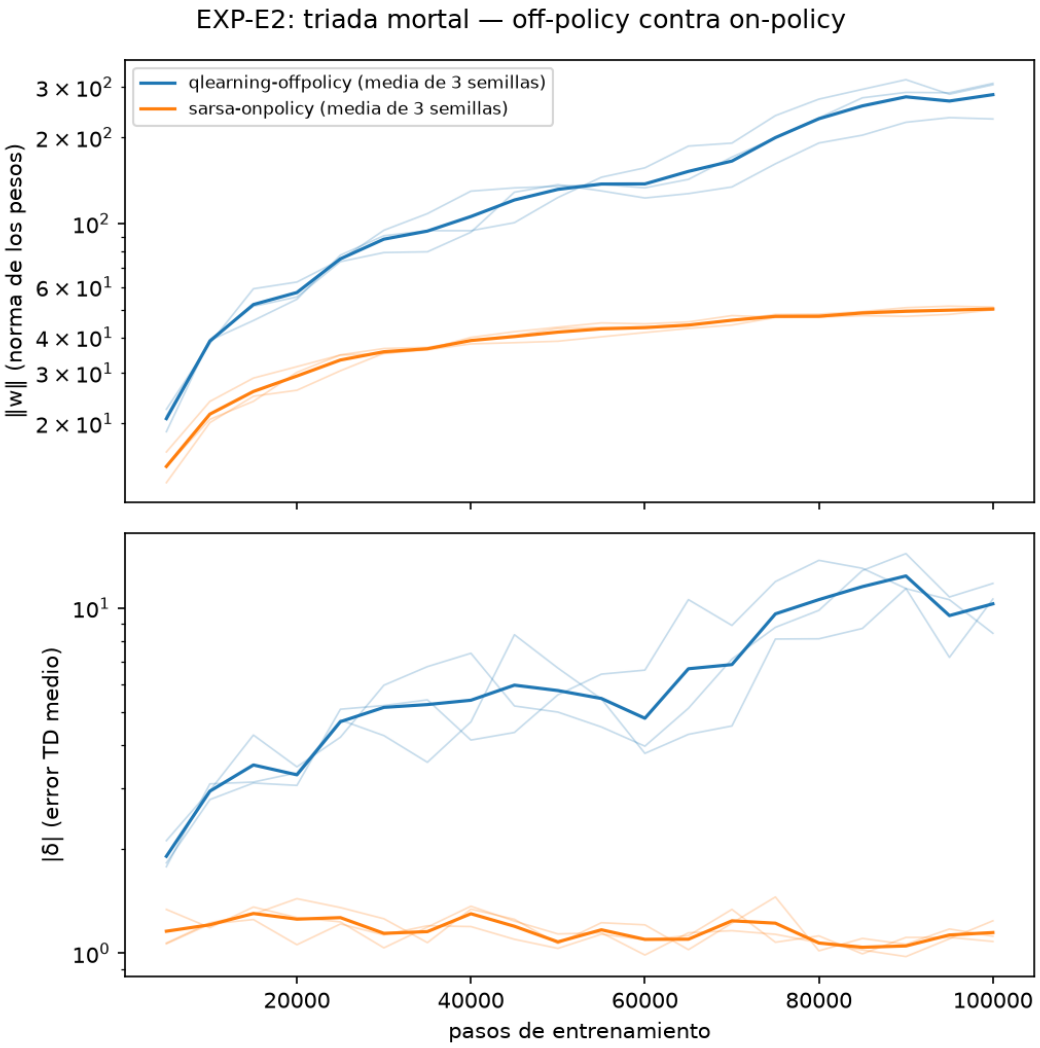


Figura 7: EXP-E2: norma de los pesos, Q-learning frente a SARSA lineal.

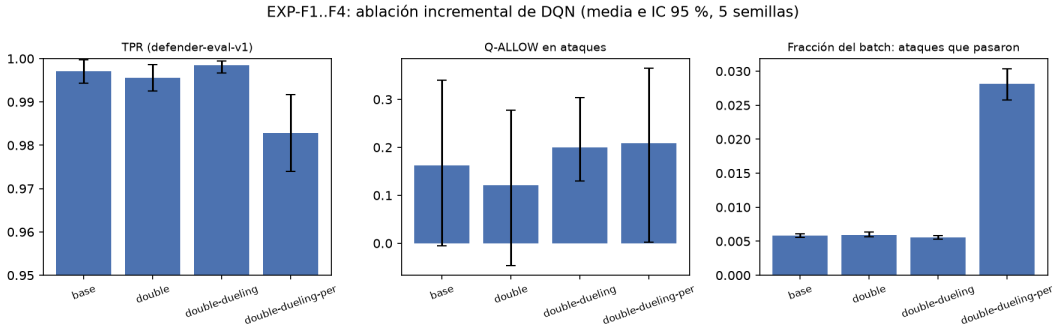


Figura 8: EXP-F1 a F4: ablación de DQN.

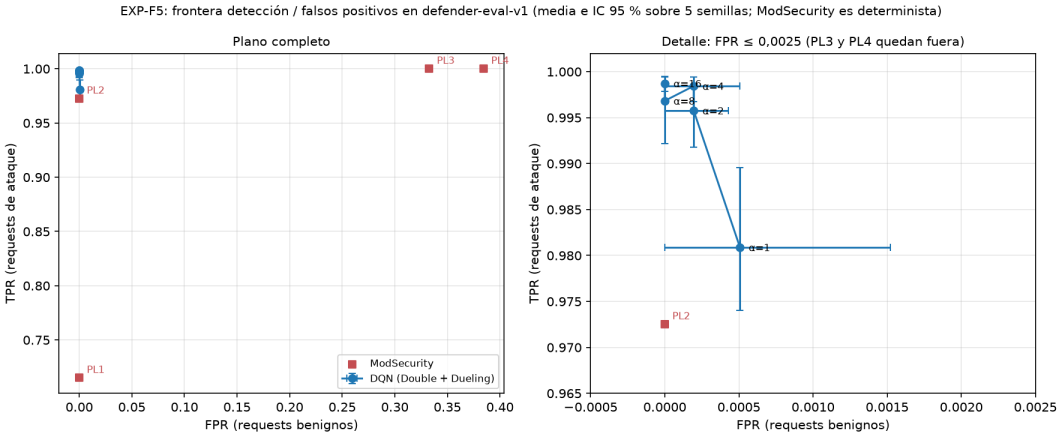


Figura 9: EXP-F5: frontera detección y falsos positivos frente a ModSecurity.

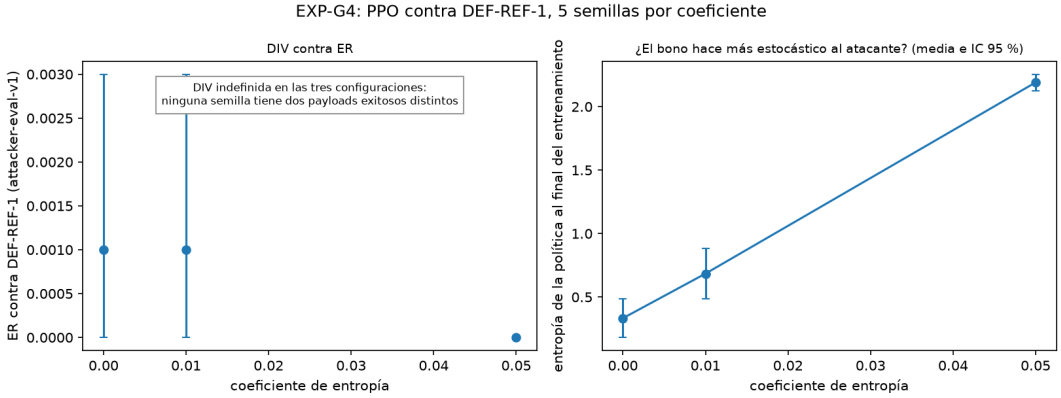


Figura 10: EXP-G4: diversidad contra evasión por bono de entropía.

6. Planning y self-play

Preguntas

Este capítulo responde RQ3 —si el self-play con un pool ayuda a conservar la robustez frente a oponentes anteriores— con EXP-I1 e I2, y RQ4 —si Dyna-Q aprovecha consultas reales mejor que el aprendizaje libre de modelo— con EXP-H1 y H2. Los resultados proceden de las filas válidas del registro experimental; las figuras y tablas se generaron a partir de los resúmenes versionados.

Diseño experimental

Atacante con modelo sustituto

En EXP-H1 el atacante aprende una aproximación supervisada de la respuesta del defensor a `SEND`. Solo las consultas reales consumen presupuesto; los pasos de planificación usan la predicción del sustituto. Se compararon Q-learning (`n = 0`), Dyna-Q (`n ∈ {5, 20, 50}`) y PPO, contra `modsec-pl1` y `DEF-REF-1`. Se ejecutaron 10 semillas para métodos tabulares y 5 para PPO. El eje de muestra son las consultas reales acumuladas, no los pasos de planificación. La evaluación usa `attacker-val-v1` (EXP-H1; commits de código `7714621` y `9d54cd2`; curvas y celdas).

EXP-H2 cambió el defensor de `modsec-pl1` a `DEF-REF-1` en el paso 100 000 y comparó Dyna-Q (`n = 20`) con Dyna-Q+ (`κ ∈ {0,01; 0,001}`), 10 semillas por configuración. La recuperación se mide desde la ER previa al cambio (EXP-H2, estados de recuperación; commit `dc698cb`).

Self-play

EXP-I1 entrenó ocho generaciones con tres semillas emparejadas (0–2) para cada condición. En `naive`, el agente siempre enfrenta al oponente actual; en `pool`, lo enfrenta con probabilidad 0,5 y el resto del tiempo muestrea oponentes históricos. Se usaron 40 episodios o sesiones por referencia y generación en `val`. La comparación principal reporta TPR por solicitud frente a `ATK-REF-0` y `ATK-REF-1`; los intentos sin solicitud emitida se reintentan y cuentan en diagnósticos, no en el denominador por solicitud (EXP-I1, entrenamiento `eea6f78`; runs, curva, figura).

EXP-I2 cruzó cada una de las ocho generaciones de defensor con cada generación de atacante del pool. Cada celda usa 200 episodios de `attacker-eval-v1` en `test`, y se repitió con tres semillas: 192 celdas, 38 400 episodios en total. La selección del campeón no usó este resultado. La ER es por episodio y no es directamente comparable con el TPR por solicitud de I1 (commit de evaluación `85524f0`; celdas, matriz, figura).

Resultados de self-play (RQ3)

En la generación 8, el pool obtuvo mayor TPR descriptivo que el self-play ingenuo frente a las dos referencias. Los intervalos *bootstrap* se calcularon sobre tres semillas, no sobre solicitudes:

Condición	TPR vs. `ATK-REF-0`	TPR vs. `ATK-REF-1`
Ingenua	0,5458 [0,3029; 0,9420]	0,7334 [0,6506; 0,8196]
Pool	0,7556 [0,6589; 0,8687]	0,8234 [0,7393; 0,8947]

Las bandas se superponen y la comparación tiene solo tres semillas. Además, ambas condiciones perdieron TPR desde el inicio; las 84 evaluaciones históricas por condición no revelan una ventaja clara (medias

descriptivas 0,9810 ingenua y 0,9841 pool, calculadas sobre EXP-I1_history.csv). Por tanto, la diferencia final favorece descriptivamente al pool, pero no demuestra una mejora robusta (EXP-I1, `naive` y `pool`; entrenamiento `eea6f78`; resumen, registro).

La matriz I2 encontró 59 evasiones en 38 400 episodios. Doce de las 64 posiciones agregadas tuvieron ER positiva. El mayor promedio fue 0,0367 [0,0250; 0,0600] para el defensor de generación 8 contra el atacante de generación 1 (22 de 600 episodios entre semillas). El cruce muestra que algunas parejas históricas conservan vulnerabilidades; no implica una tasa general de evasión alta, ni permite escoger un agente mirando el conjunto `test` (EXP-I2; matriz).

Respuesta a RQ3. Con estos entrenamientos, el pool termina con mayor TPR frente a las dos referencias, pero la cantidad de semillas, los intervalos superpuestos y la pérdida de TPR en ambas condiciones impiden concluir que el pool sea superior de forma robusta. La hipótesis H3 recibe apoyo descriptivo, no evidencia concluyente.

Resultados de Dyna (RQ4)

Contra `modsec-pl1`, todos los métodos tabulares y PPO terminan con ER = 0. Contra `DEF-REF-1`, los métodos tabulares también terminan con ER = 0 en el último punto de consultas compartido. PPO alcanza ER = 0,0105 [0,0014; 0,0225] con 1 073 consultas reales. La referencia fija `ATK-REF-1` alcanza ER = 0,0050 en 200 episodios contra `DEF-REF-1`, y 0 contra PL1; las consultas de su entrenamiento no se cuentan en EXP-H1.

Método, `run_name`	Semillas	Consultas reales medias al cierre	ER vs. PL1	ER vs. `DEF-REF-1`
Q-learning, `qlearning-defref1`	10	2 426	0,0000 [0,0000; 0,0000]	0,0000 [0,0000; 0,0000]
Dyna-Q, `dyna-5-defref1`	10	2 351	0,0000 [0,0000; 0,0000]	0,0000 [0,0000; 0,0000]
Dyna-Q, `dyna-20-defref1`	10	2 469	0,0000 [0,0000; 0,0000]	0,0000 [0,0000; 0,0000]
Dyna-Q, `dyna-50-defref1`	10	2 586	0,0000 [0,0000; 0,0000]	0,0000 [0,0000; 0,0000]
PPO, `ppo-defref1`	5	1 073	0,0000 [0,0000; 0,0000]	0,0105 [0,0014; 0,0225]

Las consultas son el promedio al último punto de evaluación compartido de cada grupo; las ER e IC son los resúmenes de la suite `attacker-val-v1` (EXP-H1, `results/summary/EXP-H1.csv`). Dyna-Q no mejora la ER frente a Q-learning: ambos empatan en cero. El menor conteo de consultas de Dyna-Q con `n = 5` no constituye una mejora útil, porque no aparece ninguna evasión que comparar.

El sustituto contra `DEF-REF-1` tuvo exactitud media aproximada de 0,9386–0,9389, pero su *recall* de la clase minoritaria al cierre quedó entre 0,0083 y 0,0155. La exactitud agregada oculta que el modelo casi no identifica respuestas minoritarias relevantes para la planificación. Estos valores corresponden a `qlearning-defref1` y `dyna-{5,20,50}-defref1` (10 semillas cada uno; EXP-H1, commits `7714621` y `9d54cd2`).

En EXP-H2, las 30 semillas (10 por método) tuvieron ER previa y posterior al cambio igual a 0. El estado fue `baseline_zero` en 30 de 30; por eso no hay tiempo de recuperación definido y no se puede comparar la adaptación de Dyna-Q+ con Dyna-Q (resumen, recuperación).

EXP-H3, Prioritized Sweeping, se recortó según el orden del plan maestro. Dado que H1 y H2 no produjeron señal de evasión para los métodos tabulares, esa comparación adicional no habría resuelto la limitación del espacio de ataque (Hallazgo de F4-T04 en el plan de fase).

Respuesta a RQ4. En este espacio de acciones, los experimentos no muestran que Dyna alcance una ER positiva con menos consultas que Q-learning. El sustituto tuvo bajo *recall* minoritario, y la ER cero hizo indefinida la comparación de recuperación de H2. La hipótesis H4 no queda respaldada por esta campaña. PPO sí halló evasiones raras contra `DEF-REF-1`, pero es una referencia adicional, no evidencia de una ventaja de planificación.

Limitaciones de la fase

- La campaña de self-play usó el mínimo de tres semillas y 40 sesiones o episodios por punto de referencia; los intervalos son amplios y no permiten inferir estabilidad entre entrenamientos.
- El defensor y el atacante comparten un espacio reducido de operadores. La ausencia de evasiones de Dyna no es una cota sobre otros operadores, WAF o aplicaciones.
- La clasificación del sustituto es dominada por la clase mayoritaria; su exactitud no resume su utilidad para planificar.
- EXP-H2 no permite medir recuperación porque la ER basal es cero.
- EXP-I2 evalúa en `test`; se informa como evaluación final descriptiva y no se usa para selección. ER se mide por episodio, mientras I1 mide TPR por solicitud.
- Los archivos de corridas crudas no se versionan; los CSV resumidos y figuras citados sí forman parte del repositorio.

Figuras del capítulo

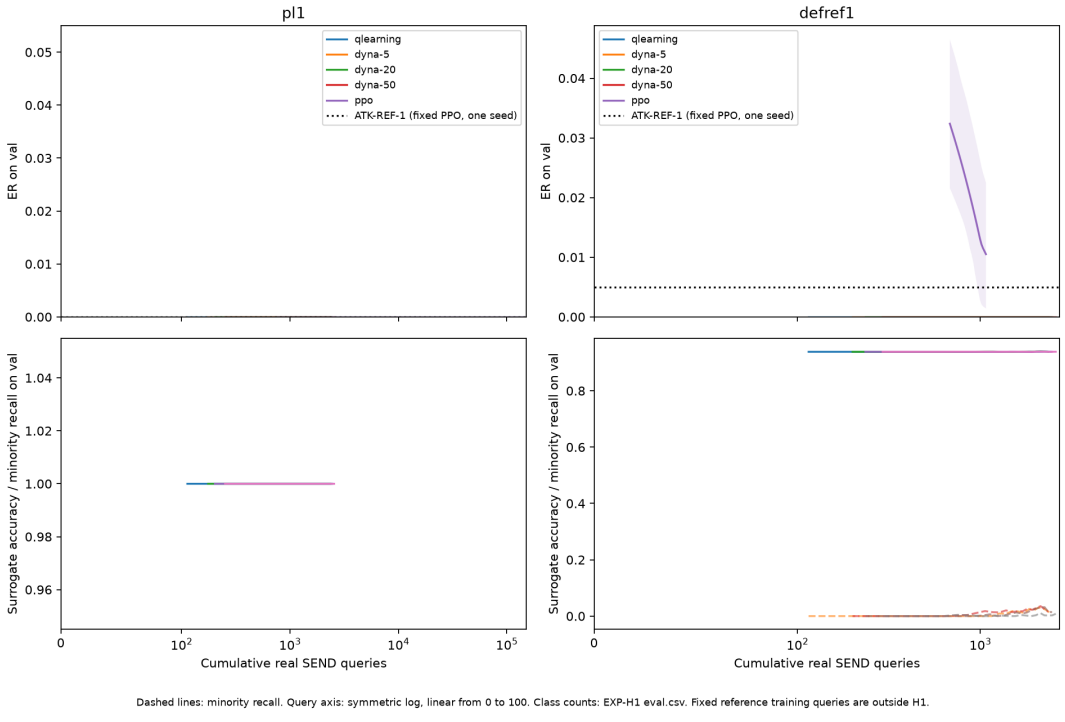


Figura 11: EXP-H1: evasión contra consultas reales.

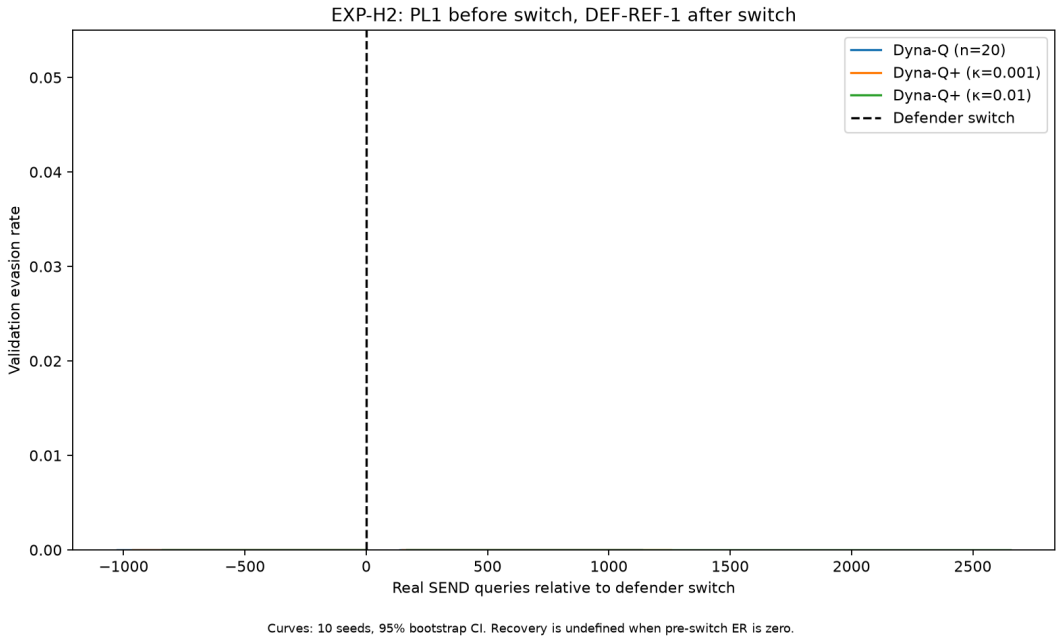


Figura 12: EXP-H2: recuperación tras el cambio de defensor.

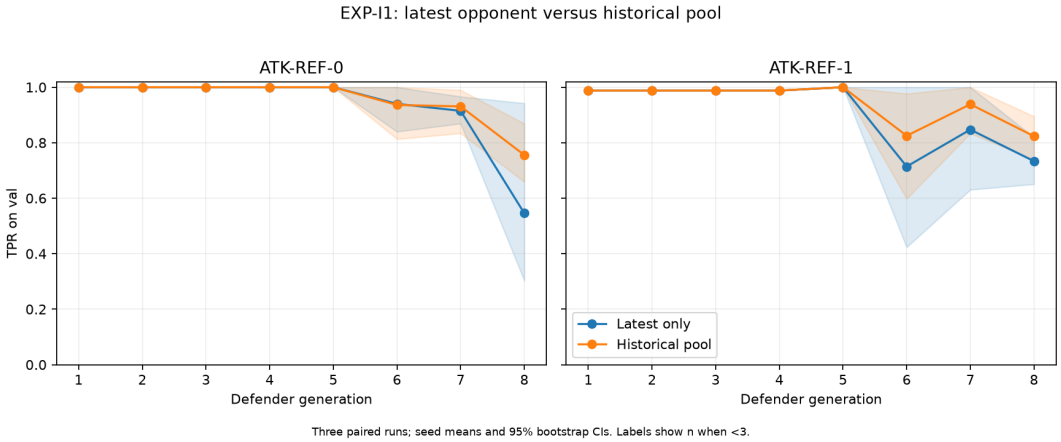


Figura 13: EXP-I1: TPR por generación, ingenuo frente a pool.

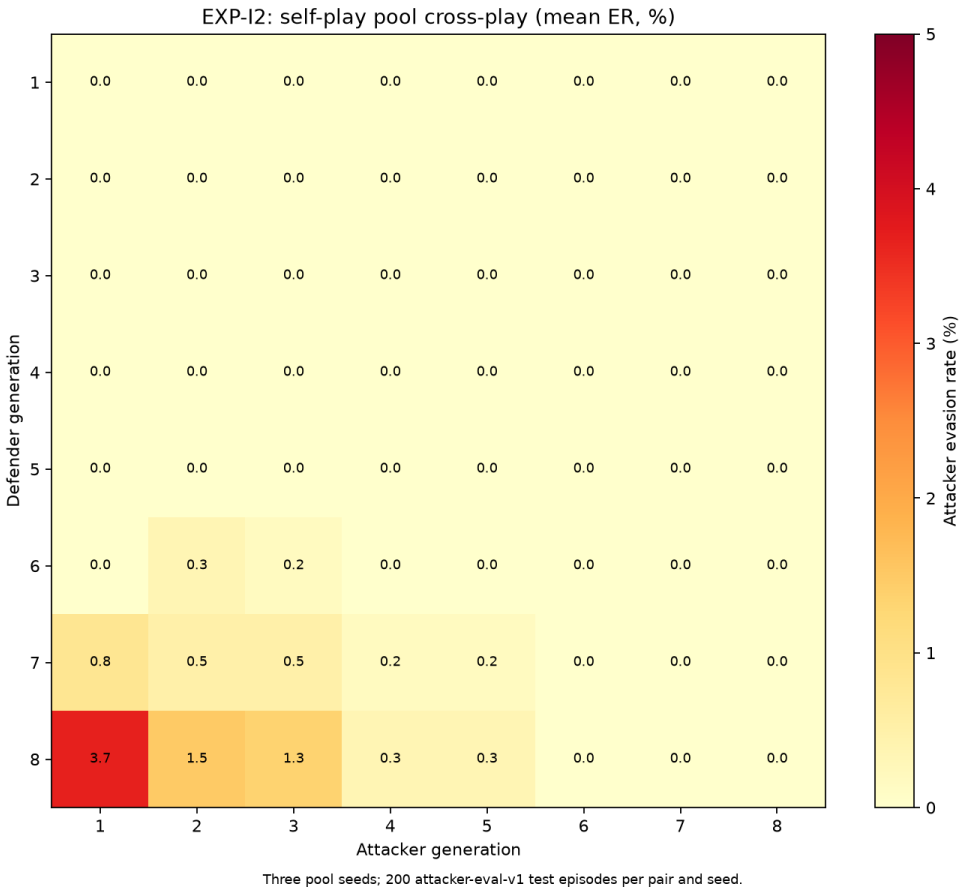


Figura 14: EXP-I2: matriz de cruces defensor por atacante.

7. Generalización

Validación del modo rápido frente al laboratorio real

EXP-R1 evaluó los checkpoints fijos de `ATK-REF-0`, `ATK-REF-1`, `DEF-REF-1` y el campeón individual de self-play (`def-gen-08`, semilla de entrenamiento 0, seleccionado únicamente con `val` en EXP-I1/F4-T08). Para cada una de tres semillas de evaluación se usaron exactamente los mismos planes `test` en `fast` y `real`: 50 episodios por atacante contra `DEF-REF-1` con un presupuesto de 50 consultas, y 500 sesiones planificadas por defensor (250 frente a cada atacante de referencia, con 450 benignas y 50 de ataque). Las identidades y hashes de pesos de los agentes, oponentes y planes se verificaron en cada par. El GAP se define como `|m_fast − m_real|` por semilla y se promedia sobre las tres semillas (EXP-R1, evaluación `c1d2f4d`; tabla completa).

Agente	Métricas definidas	GAP medio en cada métrica
`ATK-REF-0`	ER, QFE-c, BRK	0,0000
`ATK-REF-1`	ER, QFE-c, BRK	0,0000
`DEF-REF-1`	TPR, ER-d, FPR, FPR-h, FPR(s), FPR-h(s), F1, Q-ALLOW	0,0000
Campeón self-play	TPR, ER-d, FPR, FPR-h, FPR(s), FPR-h(s), F1, Q-ALLOW	0,0000

Los 22 GAP definidos fueron cero en las tres semillas; por ello sus intervalos *bootstrap* del 95 % también son `[0; 0]`. Los dos atacantes tuvieron ER = 0 en sus 150 episodios respectivos. QFE, DIV y UNQ dependen de evasiones observadas y permanecen indefinidas para ambos, en vez de recibir un cero artificial. En las semillas 0 y 2, `ATK-REF-1` no emitió ninguna solicitud en uno de los 50 intentos de ataque de cada política defensora. Estos intentos se conservaron en el conteo planificado, pero se excluyeron de métricas por solicitud; cada defensor tuvo 499 sesiones efectivas en esas semillas y 500 en la semilla 1 (celdas y denominadores). La ER del atacante y la ER-d del defensor proceden de planes distintos y no deben compararse como si fueran la misma tasa.

La coincidencia muestra que, para estos payloads y checkpoints, la aproximación rápida conservó las decisiones y métricas observadas en el laboratorio vivo. Es coherente con la implementación: `fast` reutiliza puntuaciones de ModSecurity obtenidas del mismo laboratorio y ejecuta el oráculo funcional directamente; `real` consulta ModSecurity y el oráculo por HTTP en cada evaluación. El resultado no cuantifica latencia y no extiende la equivalencia a otros payloads, configuraciones del WAF o condiciones del laboratorio. Los intervalos solo reflejan tres semillas de evaluación de checkpoints fijos, no variabilidad entre entrenamientos. La evaluación en modo real no corrige las limitaciones de generalización frente a atacantes no vistos y tráfico benigno difícil identificadas en EXP-I3–I5.

8. Ética y alcance

Este proyecto produce un atacante que aprende a evadir un WAF. Esa capacidad se acota con reglas de operación que aplican por igual a personas y agentes, detalladas en docs/00-vision/etica-y-alcance.md.

Aislamiento

Toda la experimentación ocurre en una red de Docker Compose declarada `internal: true`, sin salida a internet y sin puertos publicados. Una prueba automática (`tests/lab/test_isolation.py`) verifica, con un control positivo contra un host interno, que ningún contenedor resuelve DNS externo ni abre conexiones a direcciones externas. Los únicos objetivos válidos son `vulnapp` y los servicios del propio laboratorio.

Origen del corpus

Los payloads base son cadenas de inyección SQL clásicas y de dominio público, documentadas por la comunidad de seguridad, adaptadas a PostgreSQL y a los contextos de inyección de `vulnapp`. No se incorporan payloads obtenidos de sistemas de terceros. El origen se registra en `data/payloads/SOURCES.md`.

Propósito y política de publicación

El propósito es defensivo: *red teaming* automatizado para medir y mejorar la robustez de un WAF de reglas. No se publica un generador de evasiones empaquetado contra configuraciones de producción concretas; el material público documenta el método y los resultados agregados. Si se encontrara una debilidad relevante en el OWASP CRS, se seguiría su proceso de divulgación responsable antes de describirla en detalle, y la decisión la tomaría el equipo con el catedrático.

Limitación del oráculo en producción

En el laboratorio el oráculo sabe con certeza qué request es un ataque y si tuvo efecto. En producción no existe esa señal inmediata. El informe discute cómo se aproximaría —recompensas retrasadas, etiquetado por analistas, señales indirectas como errores de base de datos o honeypots— y no presenta el defensor como listo para producción.

9. Conclusiones

Respuestas a las preguntas de investigación

Pregunta	Respuesta respaldada por la campaña
RQ1. ¿Qué tan vulnerable es ModSecurity a métodos clásicos?	En las 32 celdas de EXP-A1–D2 y RQ1, ER fue 0 para los ocho métodos frente a PL1–PL4. Esta respuesta se limita a los 16 operadores actuales y a los ataques SQLi del laboratorio. Cap. 4, RQ1_table.csv, commit `6eb3ff1`.
RQ2. ¿Compite un defensor RL con ModSecurity?	En `defender-eval-v1`, DQN Double+Dueling alcanza TPR 0,9984 [0,9967; 0,9994] y FPR 0,0002 [0,0000; 0,0005], por encima de PL1 y PL2 en TPR con FPR cercano a cero. No domina PL3/PL4, que alcanzan TPR 1,0 pero tienen FPR 0,3327/0,3844. En benignos difíciles el FPR del defensor estático sube a 0,3385 [0,2775; 0,3996]. Cap. 5, EXP-F5.csv, EXP-F7.csv, entrenamiento `1517d8f` y `d9d7d49`.
RQ3. ¿El pool de self-play mejora la robustez?	En generación 8 el pool tuvo mayor TPR descriptivo que el entrenamiento ingenuo frente a `ATK-REF-0` y `ATK-REF-1`, pero los IC se superponen y se usaron tres semillas. La matriz de cruces conserva evasiones en algunos pares históricos. No hay evidencia suficiente para afirmar superioridad robusta. Cap. 6, EXP-I1_curve.csv, EXP-I2_matrix.csv, entrenamiento `eea6f78`, evaluación `85524f0`.
RQ4. ¿Dyna mejora la evasión por consulta?	No se observó una ER positiva en Dyna-Q ni Q-learning contra `DEF-REF-1`; Dyna tampoco produjo evasiones frente a PL1. El sustituto tuvo exactitud cercana a 0,939, pero *recall* minoritario de 0,0083–0,0155. H2 no puede comparar recuperación porque todas las ER fueron cero; H3 se recortó. La campaña no respalda una ventaja de Dyna. Cap. 6, EXP-H1.csv, EXP-H2_recovery.csv, `7714621`, `9d54cd2` y `dc698cb`.
RQ5. ¿Generalizan los agentes a condiciones no vistas y cuál es la brecha fast/real?	El self-play detecta ataques de familias no vistas con TPR 0,9470 [0,9446; 0,9520] y FPR 0,0092 [0,0012; 0,0177], pero en benignos difíciles su FPR sube a 0,3112 [0,2611; 0,4084]. Contra el PPO no visto la TPR cae a 0,6108 [0,5596; 0,6812]. La Dyna elegida tuvo TPR 1,0 solo entre solicitudes emitidas; únicamente 73 de 200 sesiones de ataque previstas emitieron solicitudes. En EXP-R1, las 22 combinaciones de métrica/agente definidas tuvieron GAP fast-real de 0 en las tres semillas; esto es específico a las suites y checkpoints estudiados. Cap. 7, RQ5_final.csv, EXP-R1_gap.csv, evaluación `bf14912` y `c1d2f4d`.

Síntesis

La campaña muestra que un defensor DQN puede separar ataques de tráfico benigno ordinario en el laboratorio y competir con los niveles bajos de ModSecurity en la suite de evaluación estándar. La brecha aparece cuando cambia el tráfico benigno: los FPR de los defensores aprendidos aumentan en `test-hard`. El entrenamiento contra adversarios propios y un pool genera políticas con respuestas diversas, pero los resultados disponibles no prueban que el pool sea consistentemente superior. Para el atacante, Dyna no resolvió el bajo poder predictivo del sustituto sobre respuestas minoritarias; con los operadores evaluados, su ER fue cero. La validación rápida y real coincidió en las métricas definidas, aunque no evaluó latencia ni generalización fuera de esas condiciones.

En conjunto, los resultados contestan las RQ dentro del laboratorio y las suites registradas. No justifican

afirmar que el defensor esté listo para producción ni que el agente atacante pueda evadir WAF de otras configuraciones.

Limitaciones consolidadas

Alcance y oráculo

- El objeto de estudio es una aplicación vulnerable propia, PostgreSQL, ModSecurity con OWASP CRS y SQLi. No se cubren otras aplicaciones, motores de base de datos, familias como XSS ni WAF de producción (ADR-0001; capítulos 1 y 3).
- El oráculo de laboratorio observa directamente la consulta y los canarios. Además, el atacante puede verificar funcionalidad sin pagar una consulta real. En producción no existe esa etiqueta inmediata ni esa ventaja; el defensor no se presenta como listo para desplegar (cap. 3, cap. 8).
- El espacio de ataque está limitado a 16 operadores. Los resultados negativos frente a ModSecurity y Dyna pueden reflejar esa limitación y no una imposibilidad general de evadir un WAF (cap. 4, cap. 6).

Datos y generalización

- Los splits congelados y las suites representan un corpus y una aplicación concretos. El buen resultado en `defender-eval-v1` no se conserva ante todo el tráfico benigno: el FPR aumenta en `defender-eval-hard-v1` para ambos defensores aprendidos (EXP-F7.csv, EXP-I4.csv).
- Excepción a las 5 semillas. La definición de terminado pide al menos 5 semillas en las configuraciones principales, o una excepción justificada. Las campañas EXP-I1, I2, I3 y I4 usan 3 semillas. La Fase 4 fija ese mínimo para el self-play porque cada semilla entrena ocho generaciones de atacante y defensor (plan de la Fase 4, F4-T06 y tabla de riesgos). No se midió ni se registró el tiempo de cómputo de cada corrida, así que aquí no se cuantifica el costo. La consecuencia está declarada en RQ3: con 3 semillas no se afirma superioridad del pool. Los tabulares de EXP-H1 y EXP-H2 sí usan 10 semillas y PPO usa 5.
- Las campañas finales de self-play y generalización usan tres semillas de defensor. Los IC describen variación entre esas semillas y no una población amplia de entrenamientos. Las referencias fijas de ModSecurity se ejecutaron como suites deterministas, sin IC de entrenamiento.
- I5 evalúa un solo checkpoint PPO nuevo (semilla de entrenamiento 5) contra una cohorte de tres defensores. Para la referencia Dyna, 127 de 200 sesiones planificadas no emitieron solicitud; el TPR condicionado a solicitudes no describe el éxito por sesión planificada (EXP-I5.csv).
- I2 usa el split `test` para evaluación de cruces. Sus resultados son descriptivos y no pueden emplearse para cambiar checkpoints, umbrales o hiperparámetros (EXP-I2_matrix.csv).

Métodos y comparación

- La exactitud del sustituto de Dyna queda dominada por la clase mayoritaria y no implica capacidad predictiva útil para la clase minoritaria. ER cero vuelve indefinida la comparación de recuperación de H2; Prioritized Sweeping (H3) se recortó según el orden aprobado (cap. 6).
- El mayor TPR de cierre del pool en I1 es descriptivo: hay tres semillas, los intervalos se superponen y ambos métodos pierden TPR respecto del inicio. No se debe convertir ese patrón en una afirmación de mejora robusta.
- ER, TPR por solicitud, sesiones planificadas y solicitudes realmente emitidas usan denominadores distintos. Las tablas de los capítulos mantienen la definición y el denominador de cada suite.

- La igualdad de las métricas fast/real en R1 no mide latencia ni acredita equivalencia con otros payloads, estados de laboratorio, checkpoints o reglas. La prueba de R1 cubrió los pares de agentes, semillas y planes declarados en el capítulo 7.

Reproducibilidad y uso

- Los resúmenes compactos y las figuras están versionados; las corridas crudas se excluyen del repositorio. La regeneración requiere las configuraciones, checkpoints fuente y servicios del laboratorio. El README documenta los puntos de entrada, y el inicio rápido se verificó desde una clonación limpia de `main` (ocho servicios del laboratorio saludables, 798 pruebas unitarias). No se reconstruyó la campaña completa desde esa clonación.
- La matriz de cruces y la demo son instrumentos de evaluación y presentación del laboratorio, no una certificación del producto ni una recomendación de despliegue.

Trabajo futuro

1. Ampliar y evaluar el espacio de mutación dentro de un laboratorio propio, conservando splits independientes y el mismo control de aislamiento.
2. Mejorar la medición de tráfico benigno difícil y estudiar la selección de defensor bajo límites de falsos positivos definidos antes de ver `test-hard`.
3. Aumentar semillas y sesiones para self-play, y evaluar generaciones frente a atacantes retenidos sin usar el conjunto de prueba para selección.
4. Mejorar la calibración y el *recall* minoritario del sustituto antes de atribuir beneficios a más pasos de planificación.
5. Medir latencia y costo operacional en una suite dedicada; la igualdad de decisiones de R1 no sustituye una medición de rendimiento.
6. Ampliar el capítulo 2 con una revisión sistemática de la literatura sobre WAF con aprendizaje automático y RL ofensivo; el actual cubre solo las fuentes verificadas.

Referencias

Se cotejaron título, autoría y año en la página editorial, el archivo oficial del congreso o el DOI enlazado. La revisión confirma los metadatos y la pertinencia general indicada; antes de la entrega académica, el equipo debe aplicar el formato bibliográfico que solicite el catedrático y comprobar que cada cita aparezca en el texto correspondiente.

Clave	Referencia verificada	Se usa para	Estado y comprobación
`sutton2018`	Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2.ª ed.). MIT Press.	Bandits, DP, MC, TD, aproximación y planificación	Verificada: página editorial MIT Press; Agente F4-T11, 2026-11-08
`demetrio2020wafamole`	Demetrio, L., Valenza, A., Costa, G., & Lagorio, G. (2020). WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning. *Proceedings of the 35th Annual ACM Symposium on Applied Computing*, 1745–1752.	Contexto de mutación y evasión de WAF	Verificada: DOI y repositorio institucional del artículo; Agente F4-T11, 2026-11-08
`mnih2015dqn`	Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature, 518*, 529–533.	DQN	Verificada: DOI/página de Nature; Agente F4-T11, 2026-11-08
`vanhasselt2016double`	van Hasselt, H., Guez, A., & Silver, D. (2016). Deep Reinforcement Learning with Double Q-Learning. *Proceedings of the AAAI Conference on Artificial Intelligence, 30*(1).	Double DQN	Verificada: registro editorial AAAI y DOI; Agente F4-T11, 2026-11-08
`wang2016dueling`	Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., & de Freitas, N. (2016). Dueling Network Architectures for Deep Reinforcement Learning. *Proceedings of the 33rd International Conference on Machine Learning*, 48, 1995–2003.	Dueling DQN	Verificada: Proceedings of Machine Learning Research; Agente F4-T11, 2026-11-08
`schaul2016per`	Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). Prioritized Experience Replay. *International Conference on Learning Representations*.	Prioritized Experience Replay	Verificada: manuscrito de ICLR/OpenReview; Agente F4-T11, 2026-11-08

Clave	Referencia verificada	Se usa para	Estado y comprobación
`schulman2016gae`	Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016). High-Dimensional Continuous Control Using Generalized Advantage Estimation. *International Conference on Learning Representations*.	Estimación GAE	Verificada: metadatos del manuscrito y versión presentada en ICLR; Agente F4-T11, 2026-11-08
`schulman2017ppo`	Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.	PPO	Verificada: registro arXiv de los autores; Agente F4-T11, 2026-11-08
`mnih2016a3c`	Mnih, V., et al. (2016). Asynchronous Methods for Deep Reinforcement Learning. *Proceedings of the 33rd International Conference on Machine Learning*, 48, 1928–1937.	Métodos actor-critic asíncronos, incluido A3C	Verificada: Proceedings of Machine Learning Research. Esta referencia sustenta A3C/actor-critic asíncrono, no se atribuye como publicación original de A2C; Agente F4-T11, 2026-11-08
`williams1992reinforce`	Williams, R. J. (1992). Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning, 8*, 229–256.	REINFORCE	Verificada: DOI y metadatos editoriales; Agente F4-T11, 2026-11-08
`sutton1990dyna`	Sutton, R. S. (1991). Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin, 2*, 160–163.	Dyna	Verificada: DOI y registro editorial. Corrección: el artículo de SIGART se publicó en 1991, no en 1990; Agente F4-T11, 2026-11-08
`huang2022cleanrl`	Huang, S., et al. (2022). CleanRL: High-quality Single-file Implementations of Deep Reinforcement Learning Algorithms. *Journal of Machine Learning Research, 23*(274), 1–18.	Contexto de implementación de deep RL	Verificada: página editorial JMLR; Agente F4-T11, 2026-11-08
`owaspcrs`	OWASP Core Rule Set. *CRS Documentation*.	ModSecurity/CRS y niveles de paranoia	Verificada: documentación oficial del proyecto; consulta 2026-11-08
`gymnasium`	Farama Foundation. *Gymnasium Documentation*.	API de entornos	Verificada: documentación oficial; consulta 2026-11-08

Clave	Referencia verificada	Se usa para	Estado y comprobación
`erdodi2021sqlq`	Erdődi, L., Sommervoll, Å. Å., & Zennaro, F. M. (2021). Simulating SQL Injection Vulnerability Exploitation Using Q-Learning Reinforcement Learning Agents. *arXiv:2101.03118*.	RL aplicado a SQLi (trabajo relacionado)	Verificada: registro arXiv (título, autoría, año). Una búsqueda indica publicación posterior en *Journal of Information Security and Applications* (DOI 10.1016/j.jisa.2021.102903); volumen y artículo no se cotejaron, por eso se cita la versión arXiv; Agente F4-T11, 2026-11-09
`silver2017alphazero`	Silver, D., et al. (2017). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *arXiv:1712.01815*.	Self-play (trabajo relacionado)	Verificada: registro arXiv; la página no indica revista ni congreso; Agente F4-T11, 2026-11-09
`bansal2018emergent`	Bansal, T., Pachocki, J., Sidor, S., Sutskever, I., & Mordatch, I. (2018). Emergent Complexity via Multi-Agent Competition. *International Conference on Learning Representations*.	Competencia multiagente (trabajo relacionado)	Verificada: registro arXiv, publicado en ICLR 2018; Agente F4-T11, 2026-11-09

Todas las claves anteriores se citan en [el capítulo 2] o en los capítulos 4 a 6 según la columna «Se usa para». Las tres últimas se agregaron y verificaron en el cierre de F4-T11.

El formato final de bibliografía será el que pida el catedrático (APA 7 si no indica otro).